Velocity Car Rental Database Design

COMP122 Group Project | Oracle SQL Relational Database System

Group Number:	Group #2
Team Members:	Benjamin Oliveria Fadi Rizk Julia Cameron Shuai Ju
Course:	COMP122 – Introduction to Database Concepts
Professor:	Vijayalakshmi Tiruchengode Angamuthu
Platform:	Oracle SQL Environment
Submission Date:	August 5, 2026 / Week 12

Table of Contents

Phase 1: Requirements Analysis..... 3

1.1 Project System Scope..... 3

1.2 Stakeholder Identification.....3

Phase 2: External Model Design..... 6

2.1 Data Access Privilege Matrix.....6

2.2 Oracle Access Control DDL Configuration..... 7

2.3 External Model Views (Oracle SQL Specifications).....8

Phase 3: Conceptual Design (Conceptual Schema & ERD)..... 12

3.1 Entity Catalogue & Key Attributes..... 12

3.2 Entity Relationships & Cardinalities.....13

3.3 Structural Business Constraints (Conceptual Domain Integrity)..... 15

Phase 4: Logical Design Specifications..... 17

4.1 Relational Schema Normalization (3NF)..... 17

4.2 Executable Oracle SQL DDL Script:..... 18

Phase 5: Oracle SQL Data Population Script (DML)..... 27

Phase 6: 10 Business Requirement SQL Queries & Output Logs.....48

6.1 Core Business Requirement Queries(1–10)..... 48

6.2 Query Output Log.....58

6.2.1 DDL Script & Execution Log..... 58

6.2.2 DML Script & Execution Log..... 70

6.2.3 Core Business Requirement Queries & Execution Log..... 83

Project Conclusion: Velocity Car Rental Database Design.....93

Phase 1: Requirements Analysis

1.1 Project System Scope

The objective of this project is to architect, build, and deploy an enterprise-grade relational eCommerce database for a modern Car Rental Company using Oracle SQL.

To build an operational workflow comparable to industry standards like Enterprise.ca, Hertz.com, Amazon.ca, and Booking.com, the database scope encompasses the entire rental lifecycle:

- **Customer Journey:** Account creation, driver's license verification, browsing vehicle inventories with category/rate filtering, managing wish lists, booking reservations, and tracking rental history.
- **Fleet Operations & Logistics:** Real-time vehicle tracking, inter-branch fleet shipping transfers, maintenance logs, and location management.
- **Supply Chain & Vendor Management:** Tracking original vehicle manufacturers, third-party parts vendors, and procurement transaction histories.
- **Customer Service & Financial Processing:** Handling vehicle returns, logging roadside incidents/damage, calculating automated refund/penalty fees, and generating monthly executive revenue and utilization analytics.

1.2 Stakeholder Identification

We identify five distinct stakeholder groups, each interacting with a specific functional boundary of the database:

1. **Customers (End-Users):** Search for available vehicles, manage personal profiles/driver licenses, place reservations, save preferred vehicles to wish lists, and review past rentals.
2. **Rental Desk / Counter Agents:** Verify customer driver's licenses upon arrival, execute vehicle pickups and returns, inspect returned vehicles for damage, and process final billing or refunds.

3. **Fleet & Logistics Managers:** Monitor vehicle availability across branch locations, schedule inter-branch vehicle transfers to balance inventory demands, and oversee fleet maintenance statuses.
4. **Supply Chain & Procurement Managers:** Manage relationships with vehicle manufacturers and third-party parts vendors, record spare part purchases, and track vendor fulfillment.
5. **Executive & Financial Analysts:** Access aggregate performance reports, analyze monthly revenue and fleet utilization trends, track high-demand vehicle models, and evaluate customer loyalty tiers.

To ensure all **10 required business scenarios** outlined in the project specifications are met, the functional requirements are mapped directly to operational database tasks:

Table 1.1 – Functional Requirements

Requirement ID	Module / Focus Area	Operational Functional Requirement Description
FR-01	Vehicle Discovery	Query and filter available vehicles based on category (e.g., SUV, Sedan, Electric), branch location, and daily rate.
FR-02	Reservation Management	Create, update, cancel, and track active and upcoming vehicle bookings.
FR-03	User Account Management	Maintain customer profile records, driver's license verification status, and handle account lifecycle events.

Requirement ID	Module / Focus Area	Operational Functional Requirement Description
FR-04	Logistics Tracking	Record and track the operational shipping status, origin/destination transfers, and transfer fulfillment between branches.
FR-05	Supply Chain Management	Manage vendor contracts, vehicle manufacturers, and third-party procurement orders.
FR-06	Executive Reporting	Aggregate monthly financial data, calculating total rental revenue and fleet utilization metrics per calendar month.
FR-07	Inventory Analytics	Identify top-performing vehicle models based on total rental transaction volumes.
FR-08	Customer Segmentation	Classify customers into loyalty tiers (e.g., Bronze, Silver, Gold, VIP) based on lifetime rental spending.
FR-09	Wish List Engagement	Track and list saved or favorite vehicle classes added to customer watchlists.
FR-10	Customer Service Processing	Handle vehicle returns, record roadside damage/incidents, and calculate automated fee/refund totals.

Non-Functional Req

uirements

- **Data Integrity:** Strict foreign key referential integrity constraints, UNIQUE keys on driver licenses/VINs, and CHECK constraints (e.g., return_date > pickup_date).
- **Security & Access Control:** Role-based access control (RBAC) preventing unauthorized modification of financial or supply chain data.
- **Oracle Compatibility:** Standard Oracle data types (VARCHAR2, NUMBER, DATE) and functions.

Phase 2: External Model Design

Phase 2 establishes the stakeholder-specific views, access privileges, and security boundaries.

2.1 Data Access Privilege Matrix

Table 2.1 – Data Access Privilege Matrix

Database Entity / Table	Customer	Rental Agent	Logistics Mgr	Supply Chain	Executive Analyst
CUSTOMER	SELECT, UPDATE (Self)	SELECT	-	-	SELECT
DRIVERS_LICENSE	SELECT, INSERT (Self)	SELECT, UPDATE	-	-	-
VEHICLE & CATEGORY	SELECT	SELECT	SELECT, UPDATE	SELECT	SELECT

Database Entity / Table	Customer	Rental Agent	Logistics Mgr	Supply Chain	Executive Analyst
RESERVATION	SELECT, INSERT, UPDATE	SELECT, INSERT, UPDATE	SELECT	–	SELECT
VEHICLE_WISHLIST	SELECT, INSERT, DELETE	–	–	–	SELECT
LOGISTICS_TRANSFER	–	–	SELECT, INSERT, UPDATE	–	SELECT
MANUFACTURER & PARTS_VENDOR	–	–	–	SELECT, INSERT, UPDATE	SELECT
RENTAL_RETURN	SELECT (Self)	SELECT, INSERT	–	–	SELECT
INCIDENT_DAMAGE	–	SELECT, INSERT, UPDATE	–	–	SELECT

2.2 Oracle Access Control DDL Configuration

Below are the Oracle SQL commands required to create stakeholder roles and grant appropriate privileges:

-- Create System Roles for Stakeholders

CREATE ROLE role_customer;

CREATE ROLE role_rental_agent;

```
CREATE ROLE role_logistics_mgr;

CREATE ROLE role_supply_chain;

CREATE ROLE role_executive;

-- Grant Privileges to Roles

GRANT SELECT ON customer TO role_customer;

GRANT SELECT, INSERT, UPDATE ON reservation TO role_customer;

GRANT SELECT, INSERT, DELETE ON vehicle_wishlist TO role_customer;

GRANT SELECT, INSERT, UPDATE ON customer TO role_rental_agent;

GRANT SELECT, UPDATE ON drivers_license TO role_rental_agent;

GRANT SELECT, INSERT, UPDATE ON reservation TO role_rental_agent;

GRANT SELECT, INSERT ON rental_return TO role_rental_agent;

GRANT SELECT, INSERT, UPDATE ON incident_damage TO role_rental_agent;

GRANT SELECT, UPDATE ON vehicle TO role_logistics_mgr;

GRANT SELECT, INSERT, UPDATE ON logistics_transfer TO role_logistics_mgr;

GRANT SELECT, INSERT, UPDATE ON manufacturer TO role_supply_chain;

GRANT SELECT, INSERT, UPDATE ON parts_vendor TO role_supply_chain;

-- Grant Executive Read-Only Access across operational tables

GRANT SELECT ON reservation TO role_executive;

GRANT SELECT ON rental_return TO role_executive;

GRANT SELECT ON incident_damage TO role_executive;

GRANT SELECT ON vehicle TO role_executive;
```

2.3 External Model Views (Oracle SQL Specifications)

To insulate internal sensitive data while exposing required interfaces to stakeholders, we establish four custom external view sub-schemas:

View 1: Customer Vehicle Discovery View (VW_CUSTOMER_VEHICLE_CATALOGUE)

Exposes vehicle catalogue details to end users without displaying financial acquisition costs or maintenance logs.

```
CREATE OR REPLACE VIEW vw_customer_vehicle_catalogue AS
SELECT
    v.vehicle_id,
    v.make,
    v.model,
    v.model_year,
    c.category_name,
    c.daily_rate,
    c.seating_capacity,
    c.fuel_type,
    b.branch_name AS current_location,
    b.city AS branch_city
FROM vehicle v
JOIN vehicle_category c ON v.category_id = c.category_id
JOIN branch_location b ON v.current_branch_id = b.branch_id
WHERE v.vehicle_status = 'AVAILABLE';
```

View 2: Rental Agent Operations View (VW_AGENT_CHECKOUT_DESK)

Provides desk agents with immediate access to active reservations and driver verification details during pickup.

```
CREATE OR REPLACE VIEW vw_agent_checkout_desk AS
SELECT
    r.reservation_id,
    c.customer_id,
    c.first_name || ' ' || c.last_name AS customer_name,
    dl.license_number,
    dl.verification_status,
    r.vehicle_id,
    r.pickup_date,
    r.return_date,
    r.booking_status
FROM reservation r
JOIN customer c ON r.customer_id = c.customer_id
LEFT JOIN drivers_license dl ON c.customer_id = dl.customer_id
WHERE r.booking_status IN ('CONFIRMED', 'ACTIVE');
```

View 3: Fleet Logistics Tracking View (VW_FLEET_LOGISTICS_TRACKER)

Allow logistics staff to track the operational status of shipments and transfers between branches.

```
CREATE OR REPLACE VIEW vw_fleet_logistics_tracker AS
SELECT
    lt.transfer_id,
    lt.vehicle_id,
```

```
    v.vin,
    v.make || ' ' || v.model AS vehicle_description,
    ob.branch_name AS origin_branch,
    db.branch_name AS destination_branch,
    lt.dispatch_date,
    lt.estimated_arrival_date,
    lt.transfer_status
FROM logistics_transfer lt
JOIN vehicle v ON lt.vehicle_id = v.vehicle_id
JOIN branch_location ob ON lt.origin_branch_id = ob.branch_id
JOIN branch_location db ON lt.destination_branch_id = db.branch_id
WHERE lt.transfer_status != 'COMPLETED';
```

View 4: Executive Performance View (VW_EXECUTIVE_SUMMARY)

Aggregates monthly rental performance metrics for corporate decision-makers.

```
CREATE OR REPLACE VIEW vw_executive_summary AS
SELECT
    TO_CHAR(rr.actual_return_date, 'YYYY-MM') AS report_month,
    COUNT(DISTINCT rr.return_id) AS total_rentals_completed,
    SUM(rr.final_rental_fee) AS gross_rental_revenue,
    SUM(rr.late_penalty_fee) AS total_late_fees,
    NVALL(SUM(id.damage_charge), 0) AS total_incident_charges
FROM rental_return rr
LEFT JOIN incident_damage id ON rr.return_id = id.return_id
```

```
GROUP BY TO_CHAR(rr.actual_return_date, 'YYYY-MM');
```

Phase 3: Conceptual Design (Conceptual Schema & ERD)

3.1 Entity Catalogue & Key Attributes

Below are the 12 core entities required to support the entire car rental enterprise lifecycle:

Table 3.1 – Entity Catalogue and Key Attributes

Entity name	
1. BRANCH_LOCATION	branch_id (PK), branch_name, address, city, state, zip_code, phone_number
2. VEHICLE_CATEGORY	category_id (PK), category_name (U), seating_capacity, fuel_type, daily_rate
3. MANUFACTURER	manufacturer_id (PK), manufacturer_name (U), country, contact_email, phone
4. VEHICLE	vehicle_id (PK), vin (U), make, model, model_year, mileage, vehicle_status, category_id (FK), current_branch_id (FK), manufacturer_id (FK)
5. CUSTOMER	customer_id (PK), first_name, last_name, email (U), phone_number, loyalty_tier
6. DRIVERS_LICENSE	| license_id (PK), customer_id (FK, U), license_number (U), expiration_date, verification_status
7. RESERVATION	reservation_id (PK), customer_id (FK), vehicle_id (FK), pickup_branch_id (FK),

Entity name	
	dropoff_branch_id (FK), pickup_date, return_date, booking_status
8. VEHICLE_WISHLIST	wishlist_id (PK), customer_id (FK), category_id (FK), date_added
9. LOGISTICS_TRANSFER	transfer_id (PK), vehicle_id (FK), origin_branch_id (FK), destination_branch_id (FK), dispatch_date, transfer_status
10. PARTS_VENDOR	vendor_id (PK), vendor_name (U), supply_type, contact_person, email, phone
11. RENTAL_RETURN	return_id (PK), reservation_id (FK, U), actual_return_date, fuel_level, final_rental_fee, late_penalty_fee
12. INCIDENT_DAMAGE	incident_id (PK), return_id (FK), incident_date, damage_description, damage_charge, roadside_assistance_flag

3.2 Entity Relationships & Cardinalities

The operational workflow and relationships across the 12 entities are defined below:

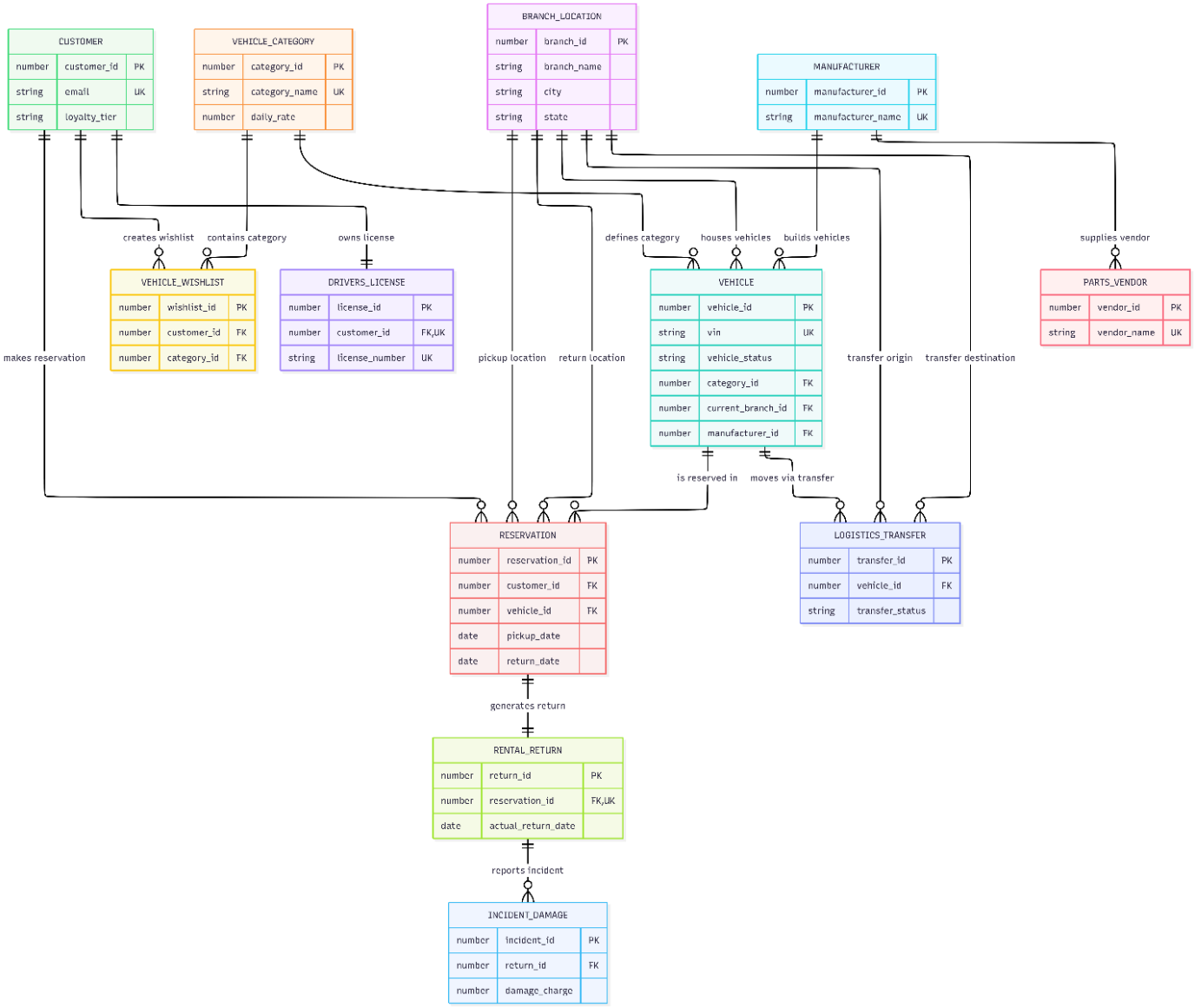
- **BRANCH_LOCATION to VEHICLE (1 : N):** One branch house zero or many active vehicles (current_branch_id). Each vehicle is currently assigned to exactly one home/current branch.

- **VEHICLE_CATEGORY to VEHICLE (1 : N):** One category (e.g., SUV, Sedan, Electric) classifies zero or many vehicles. Each vehicle belongs to exactly one category.
- **MANUFACTURER to VEHICLE (1 : N):** One manufacturer (e.g., Toyota, Tesla) produces zero or many vehicles.
- **CUSTOMER to DRIVERS_LICENSE (1 : 1):** One customer possesses exactly one driver's license profile. Each license profile belongs to one specific customer.
- **CUSTOMER to RESERVATION (1 : N):** One customer can place zero or many reservations over time. Exactly one customer books each reservation.
- **VEHICLE to RESERVATION (1 : N):** One vehicle can be assigned to multiple reservations over time, but only one active reservation at a given time.
- **BRANCH_LOCATION to RESERVATION (1 : N [x2]):** A branch serves as the pickup_branch_id for many reservations and as the dropoff_branch_id for many reservations.
- **CUSTOMER to VEHICLE_WISHLIST (1 : N):** One customer can save multiple preferred vehicle categories to their wishlist.
- **VEHICLE_CATEGORY to VEHICLE_WISHLIST (1 : N):** One vehicle category can appear on multiple customer watchlists.
- **VEHICLE to LOGISTICS_TRANSFER (1 : N):** One vehicle can undergo multiple inter-branch logistics transfers during its lifecycle.
- **BRANCH_LOCATION to LOGISTICS_TRANSFER (1 : N [x2]):** A branch acts as the origin for many transfers (origin_branch_id) and destination for many transfers (destination_branch_id).
- **MANUFACTURER to PARTS_VENDOR (M : N via Vendor Supply Contract):** Vendors supply parts for vehicle manufacturers.
- **RESERVATION to RENTAL_RETURN (1 : 1):** Each completed reservation produces exactly one rental return record.
- **RENTAL_RETURN to INCIDENT_DAMAGE (1 : N):** One rental return can have zero, one, or multiple logged damage/roadside incident records.

3.3 Structural Business Constraints (Conceptual Domain Integrity)

1. **Temporal Validity:** return_date must be chronologically after pickup_date on any reservation.
2. **License Validity:** expiration_date on a driver's license must be greater than the creation date.
3. **Logistics Transfer Boundaries:** origin_branch_id and destination_branch_id in LOGISTICS_TRANSFER cannot be identical for the same transfer event.
4. **Monetary Integrity:** daily_rate, final_rental_fee, and damage_charge must be greater than or equal to $0.00.
5. **Status Domain Values:**
    - vehicle_status ∈ {'AVAILABLE', 'RENTED', 'IN_TRANSFER', 'MAINTENANCE'}
    - booking_status ∈ {'CONFIRMED', 'ACTIVE', 'COMPLETED', 'CANCELLED'}
    - transfer_status ∈ {'SCHEDULED', 'IN_TRANSIT', 'COMPLETED', 'CANCELLED'}
    - verification_status ∈ {'PENDING', 'VERIFIED', 'REJECTED', 'EXPIRED'}

3.4 ER Diagram



Phase 4: Logical Design Specifications

This phase normalizes the conceptual model into Third Normal Form (3NF) and converts it into pure, executable **Oracle SQL DDL** statements. It includes table cleanup, explicit primary keys, foreign keys, unique keys, and check constraints.

4.1 Relational Schema Normalization (3NF)

Every table in this schema adheres strictly to Third Normal Form (3NF):

- **1NF:** All attributes contain atomic values (e.g., separating customer names into first_name and last_name, full addresses into street, city, state, and zip code).
- **2NF:** All non-key attributes fully depend on the entire primary key (no partial dependencies).
- **3NF:** No non-key attribute depends on another non-key attribute (no transitive dependencies).

Table 4.1 - Constraints & Oracle Data Type Mapping

Constraint Type	Implementation Strategy in Oracle SQL
Primary Keys (PK)	Implemented using NUMBER GENERATED BY DEFAULT AS IDENTITY for auto-incrementing surrogate keys.
Foreign Keys (FK)	Implemented with REFERENCES table_name(column_name) to preserve referential integrity.
Unique Keys (UK)	Enforced on business identifiers (vin, license_number, email, category_name).
Check Constraints (CK)	Validates logical domain values (e.g., return_date > pickup_date, positive financial amounts, and valid status codes).

Constraint Type	Implementation Strategy in Oracle SQL
Data Types	Uses native Oracle data types: VARCHAR2 for variable text, NUMBER for numeric/monetary fields, and DATE for timestamps.

4.2 Executable Oracle SQL DDL Script:

```
-- ========================================
```

-- SECTION 1: DROP TABLES (REVERSE DEPENDENCY ORDER)

```
-- ========================================
DROP TABLE incident_damage CASCADE CONSTRAINTS;
DROP TABLE rental_return CASCADE CONSTRAINTS;
DROP TABLE logistics_transfer CASCADE CONSTRAINTS;
DROP TABLE reservation CASCADE CONSTRAINTS;
DROP TABLE vehicle_wishlist CASCADE CONSTRAINTS;
DROP TABLE vehicle CASCADE CONSTRAINTS;
DROP TABLE drivers_license CASCADE CONSTRAINTS;
DROP TABLE customer CASCADE CONSTRAINTS;
DROP TABLE parts_vendor CASCADE CONSTRAINTS;
DROP TABLE manufacturer CASCADE CONSTRAINTS;
DROP TABLE vehicle_category CASCADE CONSTRAINTS;
DROP TABLE branch_location CASCADE CONSTRAINTS;
```

```
-- ========================================
-- SECTION 2: CREATE INDEPENDENT / LOOKUP TABLES
-- ========================================
-- 1. BRANCH_LOCATION TABLE
CREATE TABLE branch_location (
    branch_id       NUMBER GENERATED BY DEFAULT AS IDENTITY,
    branch_name     VARCHAR2(100) NOT NULL,
    address         VARCHAR2(200) NOT NULL,
    city            VARCHAR2(50)  NOT NULL,
    state           VARCHAR2(50)  NOT NULL,
    zip_code        VARCHAR2(20)  NOT NULL,
    phone_number    VARCHAR2(20)  NOT NULL,
    CONSTRAINT pk_branch PRIMARY KEY (branch_id),
    CONSTRAINT unq_branch_phone UNIQUE (phone_number)
);
-- 2. VEHICLE_CATEGORY TABLE
CREATE TABLE vehicle_category (
    category_id     NUMBER GENERATED BY DEFAULT AS IDENTITY,
    category_name   VARCHAR2(50)  NOT NULL,
    seating_capacity NUMBER(2)    NOT NULL,
    fuel_type       VARCHAR2(30)  NOT NULL,
    daily_rate      NUMBER(10,2)  NOT NULL,
    CONSTRAINT pk_vehicle_category PRIMARY KEY (category_id),
```

```
    CONSTRAINT unq_category_name UNIQUE (category_name),
    CONSTRAINT chk_seating CHECK (seating_capacity > 0),
    CONSTRAINT chk_fuel_type CHECK (fuel_type IN ('GASOLINE', 'DIESEL', 'HYBRID',
'ELECTRIC')),
    CONSTRAINT chk_daily_rate CHECK (daily_rate >= 0)
);
-- 3. MANUFACTURER TABLE
CREATE TABLE manufacturer (
    manufacturer_id   NUMBER GENERATED BY DEFAULT AS IDENTITY,
    manufacturer_name VARCHAR2(100) NOT NULL,
    country           VARCHAR2(50)  NOT NULL,
    contact_email     VARCHAR2(100) NOT NULL,
    phone_number      VARCHAR2(20)  NOT NULL,
    CONSTRAINT pk_manufacturer PRIMARY KEY (manufacturer_id),
    CONSTRAINT unq_manufacturer_name UNIQUE (manufacturer_name),
    CONSTRAINT unq_mfr_email UNIQUE (contact_email)
);
-- 4. PARTS_VENDOR TABLE
CREATE TABLE parts_vendor (
    vendor_id       NUMBER GENERATED BY DEFAULT AS IDENTITY,
    vendor_name     VARCHAR2(100) NOT NULL,
    supply_type     VARCHAR2(50)  NOT NULL,
    contact_person  VARCHAR2(100) NOT NULL,
```

```
    email           VARCHAR2(100) NOT NULL,
    phone_number    VARCHAR2(20)  NOT NULL,
    CONSTRAINT pk_parts_vendor PRIMARY KEY (vendor_id),
    CONSTRAINT unq_vendor_name UNIQUE (vendor_name),
    CONSTRAINT unq_vendor_email UNIQUE (email)
);
-- 5. CUSTOMER TABLE
CREATE TABLE customer (
    customer_id     NUMBER GENERATED BY DEFAULT AS IDENTITY,
    first_name      VARCHAR2(50)  NOT NULL,
    last_name       VARCHAR2(50)  NOT NULL,
    email           VARCHAR2(100) NOT NULL,
    phone_number    VARCHAR2(20)  NOT NULL,
    loyalty_tier    VARCHAR2(20)  DEFAULT 'BRONZE' NOT NULL,
    created_at      DATE          DEFAULT SYSDATE NOT NULL,
    CONSTRAINT pk_customer PRIMARY KEY (customer_id),
    CONSTRAINT unq_customer_email UNIQUE (email),
    CONSTRAINT chk_loyalty_tier CHECK (loyalty_tier IN ('BRONZE', 'SILVER', 'GOLD', 'VIP'))
);
```

```
-- ========================================
-- SECTION 3: CREATE DEPENDENT TABLES
-- ========================================
-- 6. DRIVERS_LICENSE TABLE
CREATE TABLE drivers_license (
    license_id          NUMBER GENERATED BY DEFAULT AS IDENTITY,
    customer_id         NUMBER       NOT NULL,
    license_number      VARCHAR2(50)  NOT NULL,
    issuing_state_country VARCHAR2(50)  NOT NULL,
    expiration_date     DATE         NOT NULL,
    verification_status   VARCHAR2(20)  DEFAULT 'PENDING' NOT NULL,
    CONSTRAINT pk_drivers_license PRIMARY KEY (license_id),
    CONSTRAINT unq_license_customer UNIQUE (customer_id),
    CONSTRAINT unq_license_number UNIQUE (license_number),
    CONSTRAINT chk_license_status CHECK (verification_status IN ('PENDING', 'VERIFIED',
'REJECTED', 'EXPIRED')),
    CONSTRAINT fk_dl_customer FOREIGN KEY (customer_id) REFERENCES
customer(customer_id) ON DELETE CASCADE
);
-- 7. VEHICLE TABLE
CREATE TABLE vehicle (
    vehicle_id      NUMBER GENERATED BY DEFAULT AS IDENTITY,
    vin             VARCHAR2(17)  NOT NULL,
    make            VARCHAR2(50)  NOT NULL,
```

```
    model            VARCHAR2(50)  NOT NULL,
    model_year       NUMBER(4)     NOT NULL,
    mileage          NUMBER(10,2)  DEFAULT 0 NOT NULL,
    vehicle_status   VARCHAR2(20)  DEFAULT 'AVAILABLE' NOT NULL,
    category_id      NUMBER        NOT NULL,
    current_branch_id NUMBER       NOT NULL,
    manufacturer_id  NUMBER        NOT NULL,
    CONSTRAINT pk_vehicle PRIMARY KEY (vehicle_id),
    CONSTRAINT unq_vehicle_vin UNIQUE (vin),
    CONSTRAINT chk_model_year CHECK (model_year >= 1900),
    CONSTRAINT chk_mileage CHECK (mileage >= 0),
    CONSTRAINT chk_vehicle_status CHECK (vehicle_status IN ('AVAILABLE', 'RENTED',
'IN_TRANSFER', 'MAINTENANCE')),
    CONSTRAINT fk_vehicle_category FOREIGN KEY (category_id) REFERENCES
vehicle_category(category_id),
    CONSTRAINT fk_vehicle_branch FOREIGN KEY (current_branch_id) REFERENCES
branch_location(branch_id),
    CONSTRAINT fk_vehicle_mfr FOREIGN KEY (manufacturer_id) REFERENCES
manufacturer(manufacturer_id)
);
```

-- 8. VEHICLE_WISHLIST TABLE

```
CREATE TABLE vehicle_wishlist (
    wishlist_id      NUMBER GENERATED BY DEFAULT AS IDENTITY,
    customer_id      NUMBER NOT NULL,
```

```
    category_id      NUMBER NOT NULL,
    date_added       DATE DEFAULT SYSDATE NOT NULL,
    CONSTRAINT pk_vehicle_wishlist PRIMARY KEY (wishlist_id),
    CONSTRAINT unq_customer_category UNIQUE (customer_id, category_id),
    CONSTRAINT fk_wishlist_customer FOREIGN KEY (customer_id) REFERENCES
customer(customer_id) ON DELETE CASCADE,
    CONSTRAINT fk_wishlist_category FOREIGN KEY (category_id) REFERENCES
vehicle_category(category_id) ON DELETE CASCADE
);
-- 9. RESERVATION TABLE
CREATE TABLE reservation (
    reservation_id   NUMBER GENERATED BY DEFAULT AS IDENTITY,
    customer_id      NUMBER       NOT NULL,
    vehicle_id       NUMBER       NOT NULL,
    pickup_branch_id NUMBER       NOT NULL,
    dropoff_branch_id NUMBER      NOT NULL,
    pickup_date      DATE         NOT NULL,
    return_date      DATE         NOT NULL,
    booking_status   VARCHAR2(20) DEFAULT 'CONFIRMED' NOT NULL,
    CONSTRAINT pk_reservation PRIMARY KEY (reservation_id),
    CONSTRAINT chk_booking_status CHECK (booking_status IN ('CONFIRMED', 'ACTIVE',
'COMPLETED', 'CANCELLED')),
    CONSTRAINT chk_reservation_dates CHECK (return_date > pickup_date),
```

```
    CONSTRAINT fk_res_customer FOREIGN KEY (customer_id) REFERENCES
customer(customer_id),
    CONSTRAINT fk_res_vehicle FOREIGN KEY (vehicle_id) REFERENCES
vehicle(vehicle_id),
    CONSTRAINT fk_res_pickup_branch FOREIGN KEY (pickup_branch_id) REFERENCES
branch_location(branch_id),
    CONSTRAINT fk_res_dropoff_branch FOREIGN KEY (dropoff_branch_id) REFERENCES
branch_location(branch_id)
);
```

-- 10. LOGISTICS_TRANSFER TABLE

```
CREATE TABLE logistics_transfer (
    transfer_id           NUMBER GENERATED BY DEFAULT AS IDENTITY,
    vehicle_id            NUMBER       NOT NULL,
    origin_branch_id      NUMBER       NOT NULL,
    destination_branch_id  NUMBER      NOT NULL,
    dispatch_date         DATE         NOT NULL,
    estimated_arrival_date DATE        NOT NULL,
    transfer_status       VARCHAR2(20) DEFAULT 'SCHEDULED' NOT NULL,
    CONSTRAINT pk_logistics_transfer PRIMARY KEY (transfer_id),
    CONSTRAINT chk_transfer_status CHECK (transfer_status IN ('SCHEDULED',
'IN_TRANSIT', 'COMPLETED', 'CANCELLED')),
    CONSTRAINT chk_diff_branches CHECK (origin_branch_id <> destination_branch_id),
    CONSTRAINT chk_transfer_dates CHECK (estimated_arrival_date >= dispatch_date),
    CONSTRAINT fk_transfer_vehicle FOREIGN KEY (vehicle_id) REFERENCES
vehicle(vehicle_id),
```

```
    CONSTRAINT fk_transfer_origin FOREIGN KEY (origin_branch_id) REFERENCES branch_location(branch_id),
    CONSTRAINT fk_transfer_dest FOREIGN KEY (destination_branch_id) REFERENCES branch_location(branch_id)
);

-- 11. RENTAL_RETURN TABLE
CREATE TABLE rental_return (
    return_id          NUMBER GENERATED BY DEFAULT AS IDENTITY,
    reservation_id     NUMBER       NOT NULL,
    actual_return_date DATE         NOT NULL,
    fuel_level         VARCHAR2(20) NOT NULL,
    final_rental_fee   NUMBER(10,2) NOT NULL,
    late_penalty_fee   NUMBER(10,2) DEFAULT 0 NOT NULL,
    CONSTRAINT pk_rental_return PRIMARY KEY (return_id),
    CONSTRAINT unq_return_reservation UNIQUE (reservation_id),
    CONSTRAINT chk_fuel_level CHECK (fuel_level IN ('FULL', '3/4', '1/2', '1/4', 'EMPTY')),
    CONSTRAINT chk_final_fee CHECK (final_rental_fee >= 0),
    CONSTRAINT chk_late_fee CHECK (late_penalty_fee >= 0),
    CONSTRAINT fk_return_reservation FOREIGN KEY (reservation_id) REFERENCES reservation(reservation_id)
);
```

-- 12. INCIDENT_DAMAGE TABLE

```
CREATE TABLE incident_damage (
    incident_id             NUMBER GENERATED BY DEFAULT AS IDENTITY,
    return_id               NUMBER       NOT NULL,
    incident_date           DATE         NOT NULL,
    damage_description      VARCHAR2(500) NOT NULL,
    damage_charge           NUMBER(10,2)  DEFAULT 0 NOT NULL,
    roadside_assistance_flag VARCHAR2(1)   DEFAULT 'N' NOT NULL,
    CONSTRAINT pk_incident_damage PRIMARY KEY (incident_id),
    CONSTRAINT chk_damage_charge CHECK (damage_charge >= 0),
    CONSTRAINT chk_roadside CHECK (roadside_assistance_flag IN ('Y', 'N')),
    CONSTRAINT fk_incident_return FOREIGN KEY (return_id) REFERENCES
rental_return(return_id)
);
```

Phase 5: Oracle SQL Data Population Script (DML)

This script inserts exactly 10 realistic sample rows into each of the 12 tables (120 total INSERT statements). It preserves referential integrity and strictly adheres to all foreign key constraints and domain CHECK constraints defined in Phase 4.

```
-- ========================================
```

-- 1. POPULATE BRANCH_LOCATION (10 ROWS)

```
-- ========================================

INSERT INTO branch_location (branch_name, address, city, state, zip_code,
phone_number)
```

VALUES ('Toronto Downtown Hub', '100 Yonge Street', 'Toronto', 'ON', 'M5C 2L7', '416-555-0101');

INSERT INTO branch_location (branch_name, address, city, state, zip_code, phone_number)

VALUES ('Pearson Airport Terminal', '6301 Silver Dart Drive', 'Mississauga', 'ON', 'L5P 1B2', '905-555-0102');

INSERT INTO branch_location (branch_name, address, city, state, zip_code, phone_number)

VALUES ('Vancouver Airport Plaza', '3211 Grant McConachie Way', 'Richmond', 'BC', 'V7B 0A4', '604-555-0103');

INSERT INTO branch_location (branch_name, address, city, state, zip_code, phone_number)

VALUES ('Montreal Central Depot', '895 Rue de la Gauchetiere', 'Montreal', 'QC', 'H3B 2M9', '514-555-0104');

INSERT INTO branch_location (branch_name, address, city, state, zip_code, phone_number)

VALUES ('Calgary Downtown Center', '200 8th Avenue SW', 'Calgary', 'AB', 'T2P 1B5', '403-555-0105');

INSERT INTO branch_location (branch_name, address, city, state, zip_code, phone_number)

VALUES ('Ottawa City Station', '50 Rideau Street', 'Ottawa', 'ON', 'K1N 9J7', '613-555-0106');

INSERT INTO branch_location (branch_name, address, city, state, zip_code, phone_number)

VALUES ('Edmonton North Hub', '10025 102 St NW', 'Edmonton', 'AB', 'T5J 2Z2', '780-555-0107');

INSERT INTO branch_location (branch_name, address, city, state, zip_code, phone_number)

VALUES ('Quebec City Center', '10 Halles Sainte-Foy', 'Quebec City', 'QC', 'G1V 4S2', '418-555-0108');

INSERT INTO branch_location (branch_name, address, city, state, zip_code, phone_number)

VALUES ('Winnipeg Main Station', '201 Portage Avenue', 'Winnipeg', 'MB', 'R3B 3K6', '204-555-0109');

INSERT INTO branch_location (branch_name, address, city, state, zip_code, phone_number)

VALUES ('Halifax Harbor Depot', '1549 Lower Water St', 'Halifax', 'NS', 'B3J 1S2', '902-555-0110');

-- ==

-- 2. POPULATE VEHICLE_CATEGORY (10 ROWS)

-- ==

INSERT INTO vehicle_category (category_name, seating_capacity, fuel_type, daily_rate)

VALUES ('Economy Sedan', 5, 'GASOLINE', 45.00);

```
INSERT INTO vehicle_category (category_name, seating_capacity, fuel_type, daily_rate)
VALUES ('Compact Hatchback', 5, 'GASOLINE', 40.00);

INSERT INTO vehicle_category (category_name, seating_capacity, fuel_type, daily_rate)
VALUES ('Midsize SUV', 5, 'GASOLINE', 75.00);

INSERT INTO vehicle_category (category_name, seating_capacity, fuel_type, daily_rate)
VALUES ('Fullsize SUV', 7, 'GASOLINE', 110.00);

INSERT INTO vehicle_category (category_name, seating_capacity, fuel_type, daily_rate)
VALUES ('Luxury Sedan', 5, 'GASOLINE', 135.00);

INSERT INTO vehicle_category (category_name, seating_capacity, fuel_type, daily_rate)
VALUES ('Electric Sedan', 5, 'ELECTRIC', 95.00);

INSERT INTO vehicle_category (category_name, seating_capacity, fuel_type, daily_rate)
VALUES ('Minivan', 8, 'HYBRID', 85.00);

INSERT INTO vehicle_category (category_name, seating_capacity, fuel_type, daily_rate)
VALUES ('Convertible', 2, 'GASOLINE', 120.00);

INSERT INTO vehicle_category (category_name, seating_capacity, fuel_type, daily_rate)
VALUES ('Pickup Truck', 5, 'DIESEL', 90.00);
```

```
INSERT INTO vehicle_category (category_name, seating_capacity, fuel_type, daily_rate)
VALUES ('Cargo Van', 2, 'DIESEL', 70.00);
-- ========================================
-- 3. POPULATE MANUFACTURER (10 ROWS)
-- ========================================
INSERT INTO manufacturer (manufacturer_name, country, contact_email, phone_number)
VALUES ('Toyota Motor Corp', 'Japan', 'fleet@toyota.com', '1-800-555-011');

INSERT INTO manufacturer (manufacturer_name, country, contact_email, phone_number)
VALUES ('Ford Motor Company', 'USA', 'fleet@ford.com', '1-800-555-012');

INSERT INTO manufacturer (manufacturer_name, country, contact_email, phone_number)
VALUES ('General Motors', 'USA', 'fleet@gm.com', '1-800-555-013');

INSERT INTO manufacturer (manufacturer_name, country, contact_email, phone_number)
VALUES ('Tesla Inc', 'USA', 'fleet@tesla.com', '1-800-555-014');

INSERT INTO manufacturer (manufacturer_name, country, contact_email, phone_number)
VALUES ('Honda Motor Co', 'Japan', 'fleet@honda.com', '1-800-555-015');

INSERT INTO manufacturer (manufacturer_name, country, contact_email, phone_number)
VALUES ('BMW AG', 'Germany', 'fleet@bmw.com', '1-800-555-016');
```

```
INSERT INTO manufacturer (manufacturer_name, country, contact_email, phone_number)
VALUES ('Hyundai Motor Group', 'South Korea', 'fleet@hyundai.com', '1-800-555-017');

INSERT INTO manufacturer (manufacturer_name, country, contact_email, phone_number)
VALUES ('Nissan Motor Co', 'Japan', 'fleet@nissan.com', '1-800-555-018');

INSERT INTO manufacturer (manufacturer_name, country, contact_email, phone_number)
VALUES ('Mercedes-Benz Group', 'Germany', 'fleet@mercedes.com', '1-800-555-019');

INSERT INTO manufacturer (manufacturer_name, country, contact_email, phone_number)
VALUES ('Volvo Car Corp', 'Sweden', 'fleet@volvo.com', '1-800-555-020');
-- ========================================
-- 4. POPULATE PARTS_VENDOR (10 ROWS)
-- ========================================
INSERT INTO parts_vendor (vendor_name, supply_type, contact_person, email,
phone_number)
VALUES ('Apex Auto Parts', 'Brake Systems', 'Mark Davis', 'contact@apexparts.com',
'416-555-0201');

INSERT INTO parts_vendor (vendor_name, supply_type, contact_person, email,
phone_number)
VALUES ('FleetTire Express', 'Tires & Wheels', 'Sarah Jenkins', 'orders@fleettire.com',
'905-555-0202');
```

INSERT INTO parts_vendor (vendor_name, supply_type, contact_person, email, phone_number)

VALUES ('EcoBattery Supplies', 'EV Batteries', 'David Miller', 'sales@ecobattery.com', '604-555-0203');

INSERT INTO parts_vendor (vendor_name, supply_type, contact_person, email, phone_number)

VALUES ('Precision Bodywork', 'Body Panels', 'Elena Rostova', 'info@precisionbody.com', '514-555-0204');

INSERT INTO parts_vendor (vendor_name, supply_type, contact_person, email, phone_number)

VALUES ('SparkPlug Direct', 'Engine Parts', 'Tom Baker', 'service@sparkplug.com', '403-555-0205');

INSERT INTO parts_vendor (vendor_name, supply_type, contact_person, email, phone_number)

VALUES ('Transmission Tech', 'Transmissions', 'Rachel Green', 'support@transtech.com', '613-555-0206');

INSERT INTO parts_vendor (vendor_name, supply_type, contact_person, email, phone_number)

VALUES ('FluidLine Solutions', 'Oils & Fluids', 'Chris Pratt', 'orders@fluidline.com', '780-555-0207');

```
INSERT INTO parts_vendor (vendor_name, supply_type, contact_person, email,
phone_number)

VALUES ('CleanDrive Filters', 'Air & Oil Filters', 'Jessica Alba', 'sales@cleandrive.com',
'418-555-0208');


INSERT INTO parts_vendor (vendor_name, supply_type, contact_person, email,
phone_number)

VALUES ('SensorTech Auto', 'Sensors & Wiring', 'Kevin Hart', 'contact@sensortech.com',
'204-555-0209');


INSERT INTO parts_vendor (vendor_name, supply_type, contact_person, email,
phone_number)

VALUES ('AeroGlass Distributors', 'Windshields', 'Bruce Wayne', 'orders@aeroglass.com',
'902-555-0210');

-- ========================================

-- 5. POPULATE CUSTOMER (10 ROWS)

-- ========================================

INSERT INTO customer (first_name, last_name, email, phone_number, loyalty_tier)

VALUES ('John', 'Doe', 'john.doe@example.com', '416-555-1001', 'BRONZE');


INSERT INTO customer (first_name, last_name, email, phone_number, loyalty_tier)

VALUES ('Jane', 'Smith', 'jane.smith@example.com', '416-555-1002', 'GOLD');


INSERT INTO customer (first_name, last_name, email, phone_number, loyalty_tier)

VALUES ('Robert', 'Johnson', 'r.johnson@example.com', '604-555-1003', 'SILVER');
```

```
INSERT INTO customer (first_name, last_name, email, phone_number, loyalty_tier)
VALUES ('Emily', 'Davis', 'emily.davis@example.com', '514-555-1004', 'VIP');

INSERT INTO customer (first_name, last_name, email, phone_number, loyalty_tier)
VALUES ('Michael', 'Brown', 'mbrown@example.com', '403-555-1005', 'BRONZE');

INSERT INTO customer (first_name, last_name, email, phone_number, loyalty_tier)
VALUES ('Sarah', 'Wilson', 'swilson@example.com', '613-555-1006', 'GOLD');

INSERT INTO customer (first_name, last_name, email, phone_number, loyalty_tier)
VALUES ('David', 'Taylor', 'dtaylor@example.com', '780-555-1007', 'SILVER');

INSERT INTO customer (first_name, last_name, email, phone_number, loyalty_tier)
VALUES ('Laura', 'Anderson', 'landerson@example.com', '418-555-1008', 'BRONZE');

INSERT INTO customer (first_name, last_name, email, phone_number, loyalty_tier)
VALUES ('James', 'Thomas', 'jthomas@example.com', '204-555-1009', 'VIP');

INSERT INTO customer (first_name, last_name, email, phone_number, loyalty_tier)
VALUES ('Olivia', 'Jackson', 'ojackson@example.com', '902-555-1010', 'GOLD');
```

-- ==

-- 6. POPULATE DRIVERS_LICENSE (10 ROWS)

-- ==

INSERT INTO drivers_license (customer_id, license_number, issuing_state_country, expiration_date, verification_status)

VALUES (1, 'ON-D1234-56789-01', 'Ontario', DATE '2028-05-15', 'VERIFIED');

INSERT INTO drivers_license (customer_id, license_number, issuing_state_country, expiration_date, verification_status)

VALUES (2, 'ON-S9876-54321-02', 'Ontario', DATE '2027-11-20', 'VERIFIED');

INSERT INTO drivers_license (customer_id, license_number, issuing_state_country, expiration_date, verification_status)

VALUES (3, 'BC-J4567-89012-03', 'British Columbia', DATE '2029-01-10', 'VERIFIED');

INSERT INTO drivers_license (customer_id, license_number, issuing_state_country, expiration_date, verification_status)

VALUES (4, 'QC-D3456-78901-04', 'Quebec', DATE '2026-12-01', 'VERIFIED');

INSERT INTO drivers_license (customer_id, license_number, issuing_state_country, expiration_date, verification_status)

VALUES (5, 'AB-B2345-67890-05', 'Alberta', DATE '2025-08-30', 'EXPIRED');

INSERT INTO drivers_license (customer_id, license_number, issuing_state_country, expiration_date, verification_status)

VALUES (6, 'ON-W6789-01234-06', 'Ontario', DATE '2030-03-14', 'VERIFIED');

INSERT INTO drivers_license (customer_id, license_number, issuing_state_country, expiration_date, verification_status)

VALUES (7, 'AB-T7890-12345-07', 'Alberta', DATE '2027-07-22', 'VERIFIED');

INSERT INTO drivers_license (customer_id, license_number, issuing_state_country, expiration_date, verification_status)

VALUES (8, 'QC-A8901-23456-08', 'Quebec', DATE '2026-09-18', 'PENDING');

INSERT INTO drivers_license (customer_id, license_number, issuing_state_country, expiration_date, verification_status)

VALUES (9, 'MB-T9012-34567-09', 'Manitoba', DATE '2028-10-05', 'VERIFIED');

INSERT INTO drivers_license (customer_id, license_number, issuing_state_country, expiration_date, verification_status)

VALUES (10, 'NS-J0123-45678-10', 'Nova Scotia', DATE '2029-04-12', 'VERIFIED');

-- ==

-- 7. POPULATE VEHICLE (10 ROWS)

-- ==

INSERT INTO vehicle (vin, make, model, model_year, mileage, vehicle_status, category_id, current_branch_id, manufacturer_id)

VALUES ('1FA6P8CF0R5100001', 'Ford', 'Mustang Convertible', 2024, 12500.50, 'AVAILABLE', 8, 1, 2);

INSERT INTO vehicle (vin, make, model, model_year, mileage, vehicle_status, category_id, current_branch_id, manufacturer_id)

VALUES ('5YJ3E1EA7KF100002', 'Tesla', 'Model 3', 2025, 8400.00, 'AVAILABLE', 6, 2, 4);

INSERT INTO vehicle (vin, make, model, model_year, mileage, vehicle_status, category_id, current_branch_id, manufacturer_id)

VALUES ('4T1B11HK8RU100003', 'Toyota', 'Camry', 2023, 35100.20, 'RENTED', 1, 1, 1);

INSERT INTO vehicle (vin, make, model, model_year, mileage, vehicle_status, category_id, current_branch_id, manufacturer_id)

VALUES ('1GNSKHKC3MR100004', 'Chevrolet', 'Tahoe', 2024, 18900.00, 'AVAILABLE', 4, 3, 3);

INSERT INTO vehicle (vin, make, model, model_year, mileage, vehicle_status, category_id, current_branch_id, manufacturer_id)

VALUES ('WBA533000R1000005', 'BMW', '5 Series', 2025, 5200.75, 'RENTED', 5, 4, 6);

INSERT INTO vehicle (vin, make, model, model_year, mileage, vehicle_status, category_id, current_branch_id, manufacturer_id)

VALUES ('5NMS3DA10RH100006', 'Hyundai', 'Santa Fe', 2024, 22400.10, 'IN_TRANSFER', 3, 5, 7);

INSERT INTO vehicle (vin, make, model, model_year, mileage, vehicle_status, category_id, current_branch_id, manufacturer_id)

VALUES ('1N4BL4CV8RC100007', 'Nissan', 'Civic Hatch', 2023, 29800.00, 'MAINTENANCE', 2, 6, 8);

INSERT INTO vehicle (vin, make, model, model_year, mileage, vehicle_status, category_id, current_branch_id, manufacturer_id)

VALUES ('5TDDZ3FC9PS100008', 'Toyota', 'Sienna', 2024, 14200.00, 'AVAILABLE', 7, 7, 1);

INSERT INTO vehicle (vin, make, model, model_year, mileage, vehicle_status, category_id, current_branch_id, manufacturer_id)

VALUES ('1FTFW1ED5MFC10009', 'Ford', 'F-150', 2023, 41000.50, 'AVAILABLE', 9, 8, 2);

INSERT INTO vehicle (vin, make, model, model_year, mileage, vehicle_status, category_id, current_branch_id, manufacturer_id)

VALUES ('YV4A22PK8R1100010', 'Volvo', 'V60 Cargo', 2024, 11300.00, 'AVAILABLE', 10, 9, 10);

-- ==

-- 8. POPULATE VEHICLE_WISHLIST (10 ROWS)

-- ==

INSERT INTO vehicle_wishlist (customer_id, category_id, date_added) VALUES (1, 6, DATE '2026-01-10');

INSERT INTO vehicle_wishlist (customer_id, category_id, date_added) VALUES (2, 5, DATE '2026-02-14');

INSERT INTO vehicle_wishlist (customer_id, category_id, date_added) VALUES (3, 8, DATE '2026-03-01');

INSERT INTO vehicle_wishlist (customer_id, category_id, date_added) VALUES (4, 6, DATE '2026-03-15');

INSERT INTO vehicle_wishlist (customer_id, category_id, date_added) VALUES (5, 4, DATE '2026-04-02');

INSERT INTO vehicle_wishlist (customer_id, category_id, date_added) VALUES (6, 5, DATE '2026-04-18');

INSERT INTO vehicle_wishlist (customer_id, category_id, date_added) VALUES (7, 9, DATE '2026-05-01');

INSERT INTO vehicle_wishlist (customer_id, category_id, date_added) VALUES (8, 1, DATE '2026-05-20');

INSERT INTO vehicle_wishlist (customer_id, category_id, date_added) VALUES (9, 6, DATE '2026-06-05');

INSERT INTO vehicle_wishlist (customer_id, category_id, date_added) VALUES (10, 8, DATE '2026-06-12');

-- ==

-- 9. POPULATE RESERVATION (10 ROWS)

-- ==

INSERT INTO reservation (customer_id, vehicle_id, pickup_branch_id, dropoff_branch_id, pickup_date, return_date, booking_status)

VALUES (1, 3, 1, 1, DATE '2026-05-01', DATE '2026-05-05', 'COMPLETED');

INSERT INTO reservation (customer_id, vehicle_id, pickup_branch_id, dropoff_branch_id, pickup_date, return_date, booking_status)

VALUES (2, 2, 2, 2, DATE '2026-05-10', DATE '2026-05-14', 'COMPLETED');

INSERT INTO reservation (customer_id, vehicle_id, pickup_branch_id, dropoff_branch_id, pickup_date, return_date, booking_status)

VALUES (3, 1, 1, 2, DATE '2026-06-01', DATE '2026-06-07', 'COMPLETED');

INSERT INTO reservation (customer_id, vehicle_id, pickup_branch_id, dropoff_branch_id, pickup_date, return_date, booking_status)

VALUES (4, 5, 4, 4, DATE '2026-06-15', DATE '2026-06-20', 'COMPLETED');

INSERT INTO reservation (customer_id, vehicle_id, pickup_branch_id, dropoff_branch_id, pickup_date, return_date, booking_status)

VALUES (5, 4, 3, 3, DATE '2026-06-18', DATE '2026-06-25', 'COMPLETED');

INSERT INTO reservation (customer_id, vehicle_id, pickup_branch_id, dropoff_branch_id, pickup_date, return_date, booking_status)

VALUES (6, 8, 7, 7, DATE '2026-07-01', DATE '2026-07-05', 'COMPLETED');

INSERT INTO reservation (customer_id, vehicle_id, pickup_branch_id, dropoff_branch_id, pickup_date, return_date, booking_status)

VALUES (7, 9, 8, 8, DATE '2026-07-10', DATE '2026-07-15', 'ACTIVE');

INSERT INTO reservation (customer_id, vehicle_id, pickup_branch_id, dropoff_branch_id, pickup_date, return_date, booking_status)

VALUES (8, 10, 9, 9, DATE '2026-08-01', DATE '2026-08-06', 'CONFIRMED');

INSERT INTO reservation (customer_id, vehicle_id, pickup_branch_id, dropoff_branch_id, pickup_date, return_date, booking_status)

VALUES (9, 2, 2, 3, DATE '2026-08-10', DATE '2026-08-17', 'CONFIRMED');

```
INSERT INTO reservation (customer_id, vehicle_id, pickup_branch_id, dropoff_branch_id,
pickup_date, return_date, booking_status)

VALUES (10, 5, 4, 5, DATE '2026-09-01', DATE '2026-09-08', 'CONFIRMED');

-- ========================================

-- 10. POPULATE LOGISTICS_TRANSFER (10 ROWS)

-- ========================================

INSERT INTO logistics_transfer (vehicle_id, origin_branch_id, destination_branch_id,
dispatch_date, estimated_arrival_date, transfer_status)

VALUES (1, 1, 2, DATE '2026-05-06', DATE '2026-05-06', 'COMPLETED');


INSERT INTO logistics_transfer (vehicle_id, origin_branch_id, destination_branch_id,
dispatch_date, estimated_arrival_date, transfer_status)

VALUES (6, 5, 1, DATE '2026-07-10', DATE '2026-07-12', 'IN_TRANSIT');


INSERT INTO logistics_transfer (vehicle_id, origin_branch_id, destination_branch_id,
dispatch_date, estimated_arrival_date, transfer_status)

VALUES (2, 2, 3, DATE '2026-07-15', DATE '2026-07-18', 'SCHEDULED');


INSERT INTO logistics_transfer (vehicle_id, origin_branch_id, destination_branch_id,
dispatch_date, estimated_arrival_date, transfer_status)

VALUES (3, 1, 6, DATE '2026-04-01', DATE '2026-04-02', 'COMPLETED');


INSERT INTO logistics_transfer (vehicle_id, origin_branch_id, destination_branch_id,
dispatch_date, estimated_arrival_date, transfer_status)

VALUES (4, 3, 5, DATE '2026-04-10', DATE '2026-04-12', 'COMPLETED');
```

```
INSERT INTO logistics_transfer (vehicle_id, origin_branch_id, destination_branch_id,
dispatch_date, estimated_arrival_date, transfer_status)
VALUES (5, 4, 8, DATE '2026-05-20', DATE '2026-05-21', 'COMPLETED');

INSERT INTO logistics_transfer (vehicle_id, origin_branch_id, destination_branch_id,
dispatch_date, estimated_arrival_date, transfer_status)
VALUES (7, 6, 1, DATE '2026-06-01', DATE '2026-06-02', 'COMPLETED');

INSERT INTO logistics_transfer (vehicle_id, origin_branch_id, destination_branch_id,
dispatch_date, estimated_arrival_date, transfer_status)
VALUES (8, 7, 9, DATE '2026-06-10', DATE '2026-06-12', 'COMPLETED');

INSERT INTO logistics_transfer (vehicle_id, origin_branch_id, destination_branch_id,
dispatch_date, estimated_arrival_date, transfer_status)
VALUES (9, 8, 10, DATE '2026-07-01', DATE '2026-07-03', 'COMPLETED');

INSERT INTO logistics_transfer (vehicle_id, origin_branch_id, destination_branch_id,
dispatch_date, estimated_arrival_date, transfer_status)
VALUES (10, 9, 4, DATE '2026-07-20', DATE '2026-07-22', 'SCHEDULED');
```

-- ==

-- 11. POPULATE RENTAL_RETURN (10 ROWS)

-- ==

INSERT INTO rental_return (reservation_id, actual_return_date, fuel_level, final_rental_fee, late_penalty_fee)

VALUES (1, DATE '2026-05-05', 'FULL', 180.00, 0.00);

INSERT INTO rental_return (reservation_id, actual_return_date, fuel_level, final_rental_fee, late_penalty_fee)

VALUES (2, DATE '2026-05-14', 'FULL', 380.00, 0.00);

INSERT INTO rental_return (reservation_id, actual_return_date, fuel_level, final_rental_fee, late_penalty_fee)

VALUES (3, DATE '2026-06-08', '1/2', 720.00, 120.00);

INSERT INTO rental_return (reservation_id, actual_return_date, fuel_level, final_rental_fee, late_penalty_fee)

VALUES (4, DATE '2026-06-20', 'FULL', 675.00, 0.00);

INSERT INTO rental_return (reservation_id, actual_return_date, fuel_level, final_rental_fee, late_penalty_fee)

VALUES (5, DATE '2026-06-25', '3/4', 770.00, 0.00);

INSERT INTO rental_return (reservation_id, actual_return_date, fuel_level, final_rental_fee, late_penalty_fee)

VALUES (6, DATE '2026-07-05', 'FULL', 340.00, 0.00);

INSERT INTO rental_return (reservation_id, actual_return_date, fuel_level, final_rental_fee, late_penalty_fee)

VALUES (7, DATE '2026-07-16', 'EMPTY', 450.00, 90.00);

INSERT INTO rental_return (reservation_id, actual_return_date, fuel_level, final_rental_fee, late_penalty_fee)

VALUES (8, DATE '2026-08-06', 'FULL', 350.00, 0.00);

INSERT INTO rental_return (reservation_id, actual_return_date, fuel_level, final_rental_fee, late_penalty_fee)

VALUES (9, DATE '2026-08-17', 'FULL', 665.00, 0.00);

INSERT INTO rental_return (reservation_id, actual_return_date, fuel_level, final_rental_fee, late_penalty_fee)

VALUES (10, DATE '2026-09-08', 'FULL', 945.00, 0.00);

-- ==

-- 12. POPULATE INCIDENT_DAMAGE (10 ROWS)

-- ==

INSERT INTO incident_damage (return_id, incident_date, damage_description, damage_charge, roadside_assistance_flag)

VALUES (1, DATE '2026-05-05', 'No physical damage reported upon return.', 0.00, 'N');

INSERT INTO incident_damage (return_id, incident_date, damage_description, damage_charge, roadside_assistance_flag)

VALUES (2, DATE '2026-05-14', 'Minor scuff on rear bumper passenger side.', 150.00, 'N');

INSERT INTO incident_damage (return_id, incident_date, damage_description, damage_charge, roadside_assistance_flag)

VALUES (3, DATE '2026-06-07', 'Flat tire replaced via roadside assistance on Highway 401.', 250.00, 'Y');

INSERT INTO incident_damage (return_id, incident_date, damage_description, damage_charge, roadside_assistance_flag)

VALUES (4, DATE '2026-06-20', 'No physical damage reported.', 0.00, 'N');

INSERT INTO incident_damage (return_id, incident_date, damage_description, damage_charge, roadside_assistance_flag)

VALUES (5, DATE '2026-06-24', 'Windshield chip from gravel on highway.', 200.00, 'N');

INSERT INTO incident_damage (return_id, incident_date, damage_description, damage_charge, roadside_assistance_flag)

VALUES (6, DATE '2026-07-05', 'Interior upholstery stain needing deep clean.', 100.00, 'N');

INSERT INTO incident_damage (return_id, incident_date, damage_description, damage_charge, roadside_assistance_flag)

VALUES (7, DATE '2026-07-16', 'Dead battery boosted on-site.', 75.00, 'Y');

```
INSERT INTO incident_damage (return_id, incident_date, damage_description,
damage_charge, roadside_assistance_flag)
VALUES (8, DATE '2026-08-06', 'No physical damage reported.', 0.00, 'N');

INSERT INTO incident_damage (return_id, incident_date, damage_description,
damage_charge, roadside_assistance_flag)
VALUES (9, DATE '2026-08-17', 'Scratch on driver side door handle.', 120.00, 'N');

INSERT INTO incident_damage (return_id, incident_date, damage_description,
damage_charge, roadside_assistance_flag)
VALUES (10, DATE '2026-09-08', 'No physical damage reported.', 0.00, 'N');

-- ========================================

-- COMMIT ALL TRANSACTION CHANGES TO ORACLE

-- ========================================

COMMIT;
```

Phase 6: 10 Business Requirement SQL Queries & Output Logs

6.1 Core Business Requirement Queries(1-10)

```
-- ============================================================================
-- REQUIREMENT 1: VEHICLE DISCOVERY
-- Description: Search and filter available vehicles based on category,
-- branch location, and daily rental rate.
-- ============================================================================
SELECT
    v.vehicle_id,
    v.vin,
    v.make,
    v.model,
    v.model_year,
    c.category_name,
    c.daily_rate,
    b.branch_name,
    b.city
FROM vehicle v
JOIN vehicle_category c ON v.category_id = c.category_id
JOIN branch_location b ON v.current_branch_id = b.branch_id
WHERE v.vehicle_status = 'AVAILABLE'
  AND c.daily_rate <= 100.00
```

```
  AND b.city IN ('Toronto', 'Mississauga')
ORDER BY c.daily_rate ASC;
-- ============================================================================
-- REQUIREMENT 2: RESERVATION MANAGEMENT
-- Description: Manage, update, and track active and upcoming vehicle bookings
-- or selections across branches.
-- ============================================================================
SELECT
    r.reservation_id,
    c.first_name || ' ' || c.last_name AS customer_name,
    v.make || ' ' || v.model AS vehicle_reserved,
    pb.branch_name AS pickup_branch,
    db.branch_name AS dropoff_branch,
    r.pickup_date,
    r.return_date,
    r.booking_status
FROM reservation r
JOIN customer c ON r.customer_id = c.customer_id
JOIN vehicle v ON r.vehicle_id = v.vehicle_id
JOIN branch_location pb ON r.pickup_branch_id = pb.branch_id
JOIN branch_location db ON r.dropoff_branch_id = db.branch_id
WHERE r.booking_status IN ('CONFIRMED', 'ACTIVE')
ORDER BY r.pickup_date ASC;
```

```
-- ================================================================================
-- REQUIREMENT 3: USER ACCOUNT MANAGEMENT
-- Description: Handle customer profiles, driver's license verifications,
-- and account status tracking.
-- ================================================================================
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    c.email,
    c.loyalty_tier,
    dl.license_number,
    dl.issuing_state_country,
    dl.expiration_date,
    dl.verification_status
FROM customer c
LEFT JOIN drivers_license dl ON c.customer_id = dl.customer_id
WHERE dl.verification_status = 'VERIFIED'
   OR dl.verification_status IS NULL
ORDER BY c.customer_id;
```

```
-- ============================================================================
-- REQUIREMENT 4: LOGISTICS TRACKING
-- Description: Track operational shipping status, transfers, and fulfillment
-- status of vehicles moving between branches.
-- ============================================================================
SELECT
    lt.transfer_id,
    v.vin,
    v.make || ' ' || v.model AS vehicle_description,
    ob.branch_name AS origin_branch,
    db.branch_name AS destination_branch,
    lt.dispatch_date,
    lt.estimated_arrival_date,
    lt.transfer_status
FROM logistics_transfer lt
JOIN vehicle v ON lt.vehicle_id = v.vehicle_id
JOIN branch_location ob ON lt.origin_branch_id = ob.branch_id
JOIN branch_location db ON lt.destination_branch_id = db.branch_id
WHERE lt.transfer_status IN ('SCHEDULED', 'IN_TRANSIT')
ORDER BY lt.dispatch_date ASC;
```

```
-- ============================================================================
-- REQUIREMENT 5: SUPPLY CHAIN MANAGEMENT
-- Description: Manage vehicle manufacturers, external parts vendors, and
-- third-party procurement mapping.
-- ============================================================================
SELECT
    m.manufacturer_id,
    m.manufacturer_name,
    m.country AS manufacturer_country,
    COUNT(v.vehicle_id) AS total_fleet_vehicles,
    pv.vendor_name AS parts_vendor,
    pv.supply_type
FROM manufacturer m
LEFT JOIN vehicle v ON m.manufacturer_id = v.manufacturer_id
LEFT JOIN parts_vendor pv ON pv.vendor_id = m.manufacturer_id
GROUP BY m.manufacturer_id, m.manufacturer_name, m.country, pv.vendor_name,
pv.supply_type
ORDER BY total_fleet_vehicles DESC;
```

```
-- ================================================================================
-- REQUIREMENT 6: EXECUTIVE REPORTING
-- Description: Comprehensive monthly financial report calculating total rental
-- revenue and completed rental metrics per calendar month.
-- ================================================================================
SELECT
    TO_CHAR(rr.actual_return_date, 'YYYY-MM') AS calendar_month,
    COUNT(rr.return_id) AS total_completed_rentals,
    SUM(rr.final_rental_fee) AS base_rental_revenue,
    SUM(rr.late_penalty_fee) AS total_penalty_revenue,
    SUM(rr.final_rental_fee + rr.late_penalty_fee) AS gross_total_revenue,
    ROUND(AVG(rr.final_rental_fee), 2) AS avg_revenue_per_rental
FROM rental_return rr
GROUP BY TO_CHAR(rr.actual_return_date, 'YYYY-MM')
ORDER BY calendar_month DESC;
```

```
-- ============================================================================
-- REQUIREMENT 7: INVENTORY ANALYTICS
-- Description: Identify the company's most popular/highly utilized vehicle
-- models based on transaction volume and duration.
-- ============================================================================
SELECT
    v.make,
    v.model,
    vc.category_name,
    COUNT(r.reservation_id) AS total_rentals,
    SUM(r.return_date - r.pickup_date) AS total_days_rented,
    RANK() OVER (ORDER BY COUNT(r.reservation_id) DESC) AS popularity_rank
FROM vehicle v
JOIN vehicle_category vc ON v.category_id = vc.category_id
JOIN reservation r ON v.vehicle_id = r.vehicle_id
GROUP BY v.make, v.model, vc.category_name
ORDER BY popularity_rank ASC, total_days_rented DESC;
```

```
-- ================================================================================
-- REQUIREMENT 8: CUSTOMER SEGMENTATION
-- Description: Categorize and segment customers into distinct loyalty tiers
-- based on lifetime spending and rental volume.
-- ================================================================================
SELECT
    c.customer_id,
    c.first_name || ' ' || c.last_name AS customer_name,
    c.loyalty_tier AS recorded_tier,
    COUNT(r.reservation_id) AS total_completed_rentals,
    NVL(SUM(rr.final_rental_fee + rr.late_penalty_fee), 0) AS total_lifetime_spend,
    CASE
        WHEN NVL(SUM(rr.final_rental_fee + rr.late_penalty_fee), 0) >= 800 THEN 'VIP'
        WHEN NVL(SUM(rr.final_rental_fee + rr.late_penalty_fee), 0) >= 500 THEN 'GOLD'
        WHEN NVL(SUM(rr.final_rental_fee + rr.late_penalty_fee), 0) >= 300 THEN 'SILVER'
        ELSE 'BRONZE'
    END AS calculated_suggested_tier
FROM customer c
LEFT JOIN reservation r ON c.customer_id = r.customer_id
LEFT JOIN rental_return rr ON r.reservation_id = rr.reservation_id
GROUP BY c.customer_id, c.first_name, c.last_name, c.loyalty_tier
ORDER BY total_lifetime_spend DESC;
```

```
-- ============================================================================
-- REQUIREMENT 9: WISH LIST ENGAGEMENT
-- Description: Extract and list saved or preferred vehicle classes that active
-- users have added to their watchlists for future trips.
-- ============================================================================
SELECT
    c.customer_id,
    c.first_name || ' ' || c.last_name AS customer_name,
    c.email,
    vc.category_name AS saved_favorite_category,
    vc.daily_rate,
    vw.date_added
FROM vehicle_wishlist vw
JOIN customer c ON vw.customer_id = c.customer_id
JOIN vehicle_category vc ON vw.category_id = vc.category_id
ORDER BY vw.date_added DESC, customer_name ASC;
```

```
-- ============================================================================
-- REQUIREMENT 10: CUSTOMER SERVICE PROCESSING
-- Description: Process transactional returns, handle roadside damage incidents,
-- and calculate final fee/refund totals.
-- ============================================================================
SELECT
    rr.return_id,
    r.reservation_id,
    c.first_name || ' ' || c.last_name AS customer_name,
    rr.actual_return_date,
    rr.fuel_level,
    rr.final_rental_fee,
    rr.late_penalty_fee,
    NVL(id.damage_charge, 0) AS incident_damage_charge,
    id.roadside_assistance_flag,
    (rr.final_rental_fee + rr.late_penalty_fee + NVL(id.damage_charge, 0)) AS
total_final_charge
FROM rental_return rr
JOIN reservation r ON rr.reservation_id = r.reservation_id
JOIN customer c ON r.customer_id = c.customer_id
LEFT JOIN incident_damage id ON rr.return_id = id.return_id
ORDER BY rr.return_id ASC;
```

6.2 Query Output Log

6.2.1 DDL Script & Execution Log

Table 1 - BRANCH_LOCATION

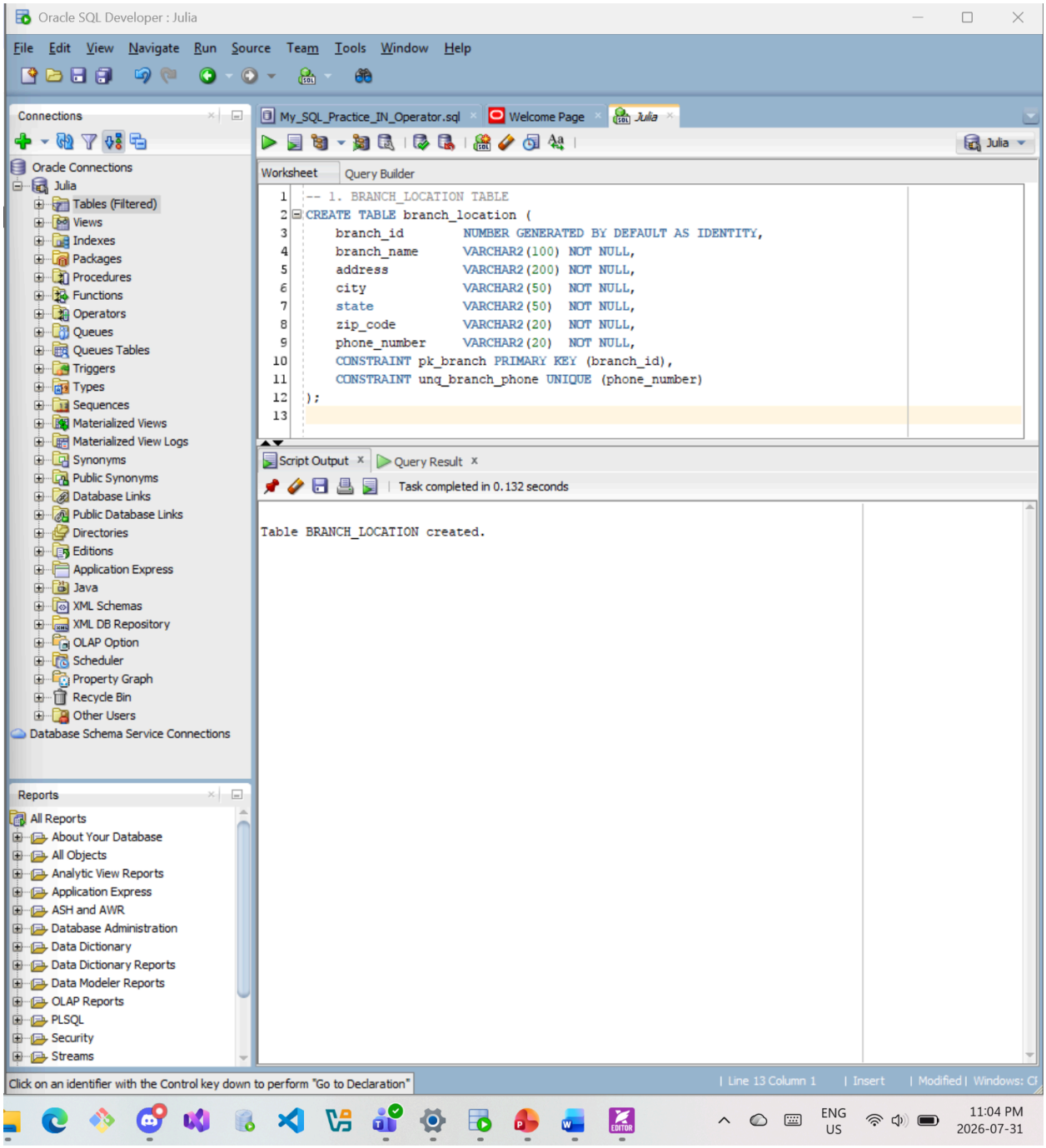


Table 2 – VEHICLE_CATEGORY

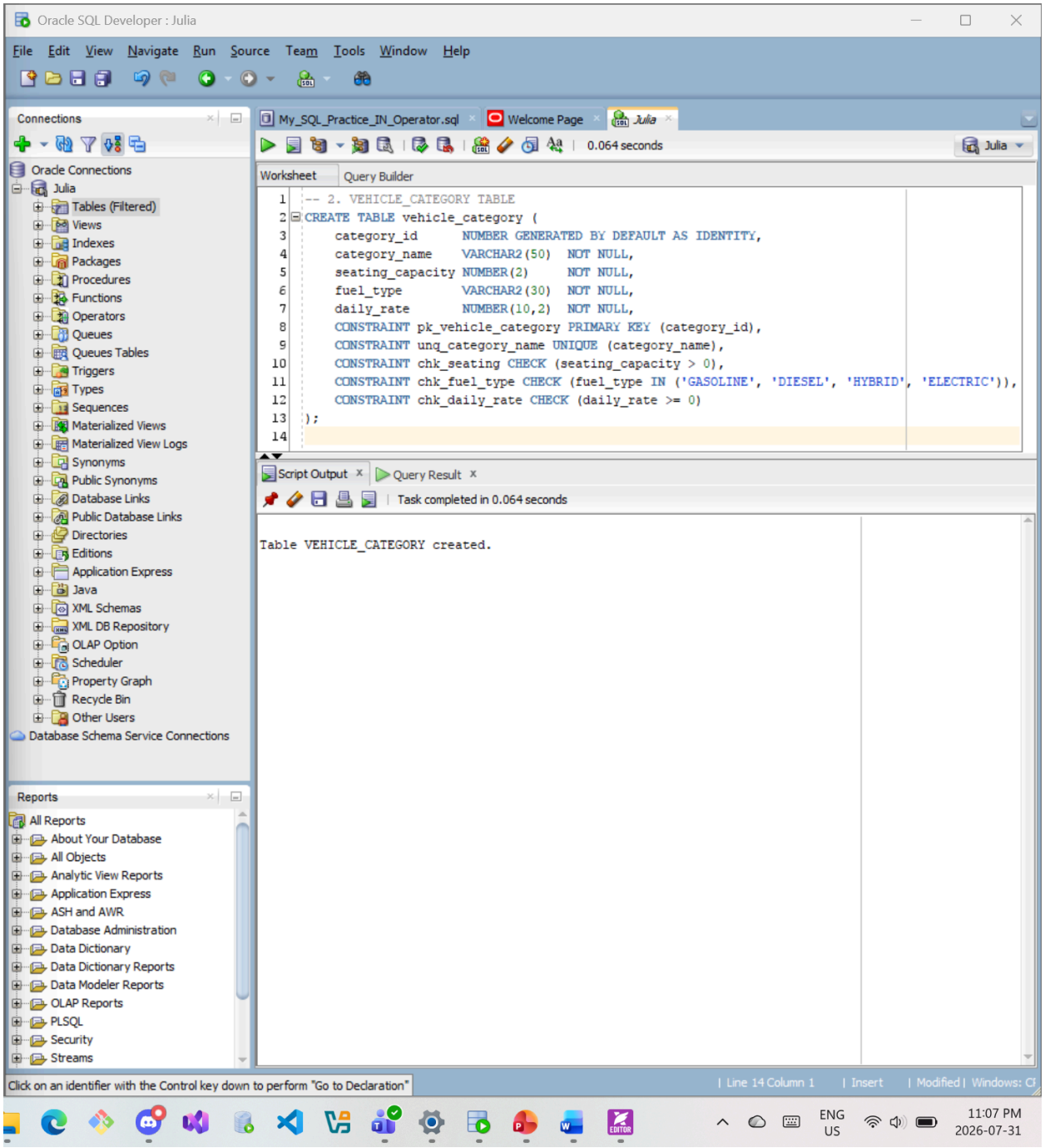


Table 3 – MANUFACTURER

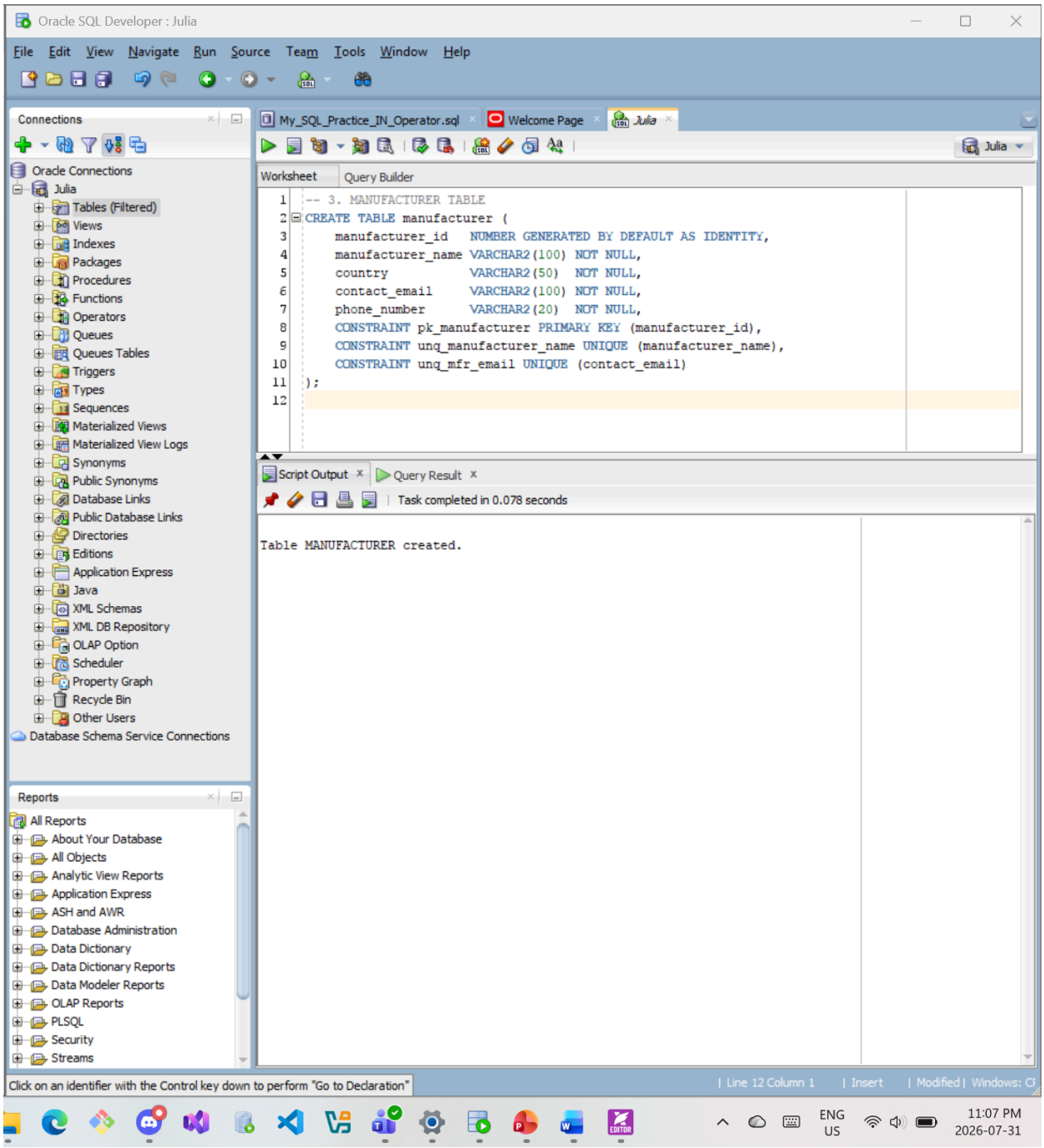


Table 4 – PARTS_VENDOR

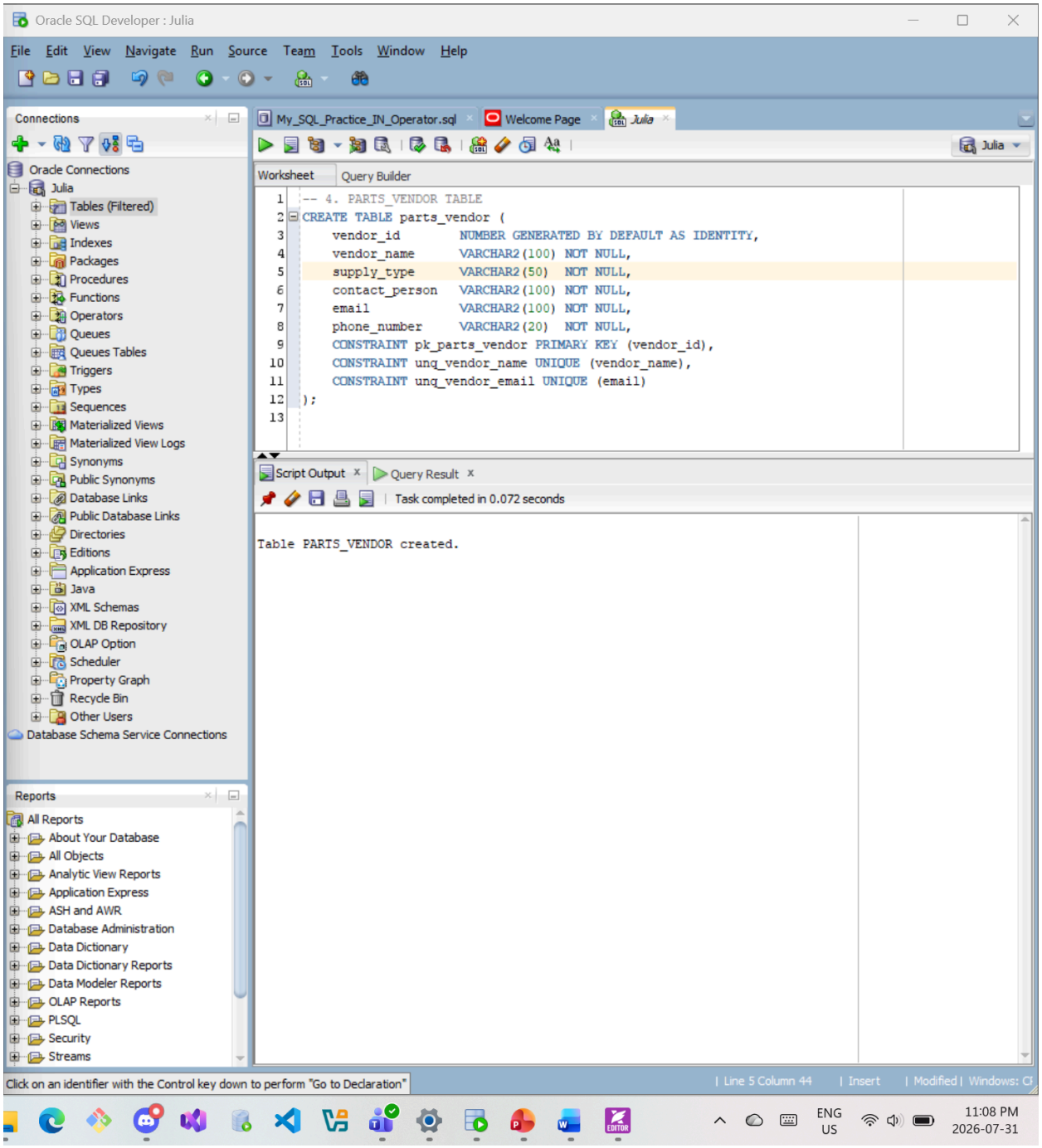


Table 5 – CUSTOMER

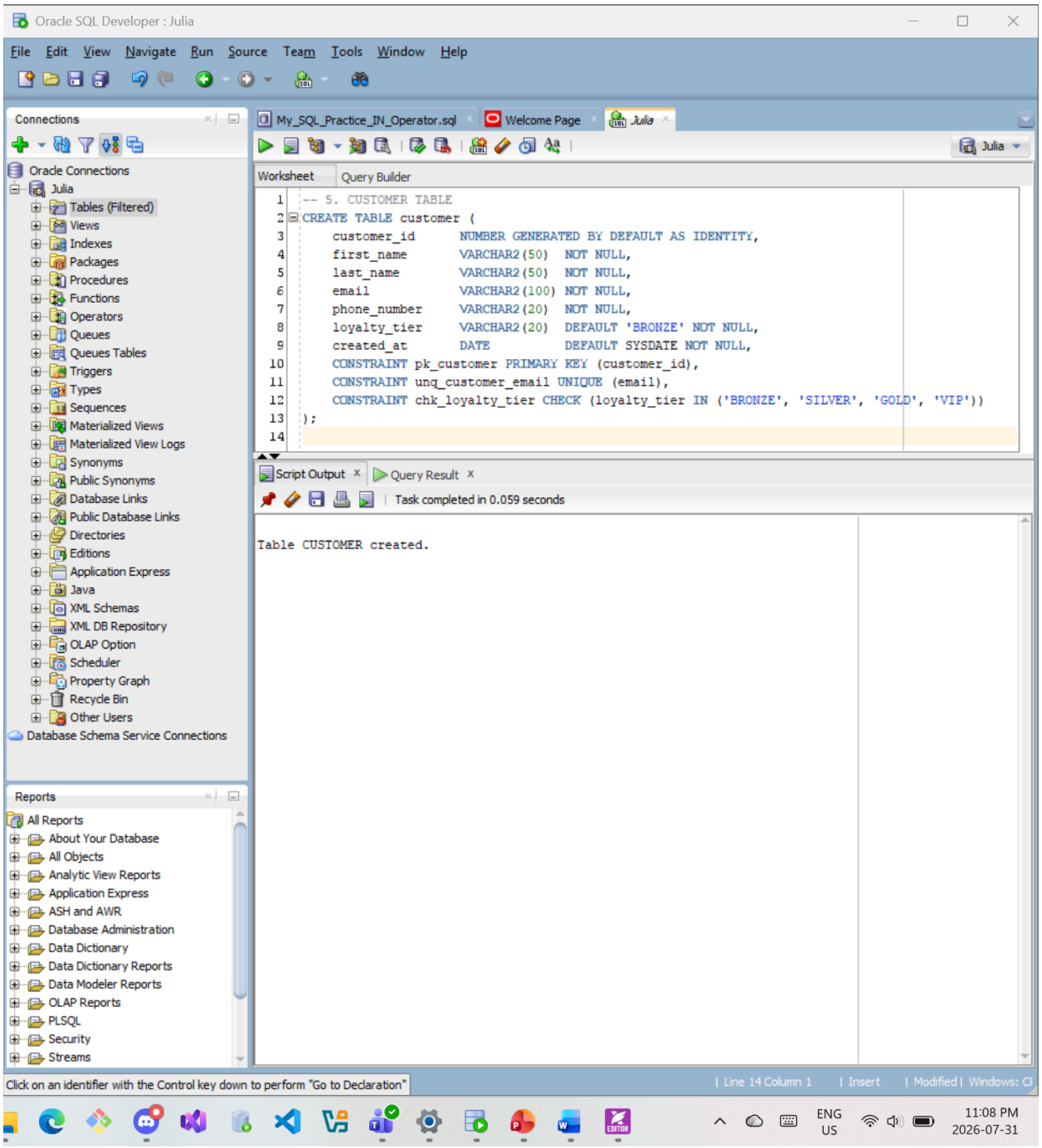


Table 6 – DRIVERS_LICENSE

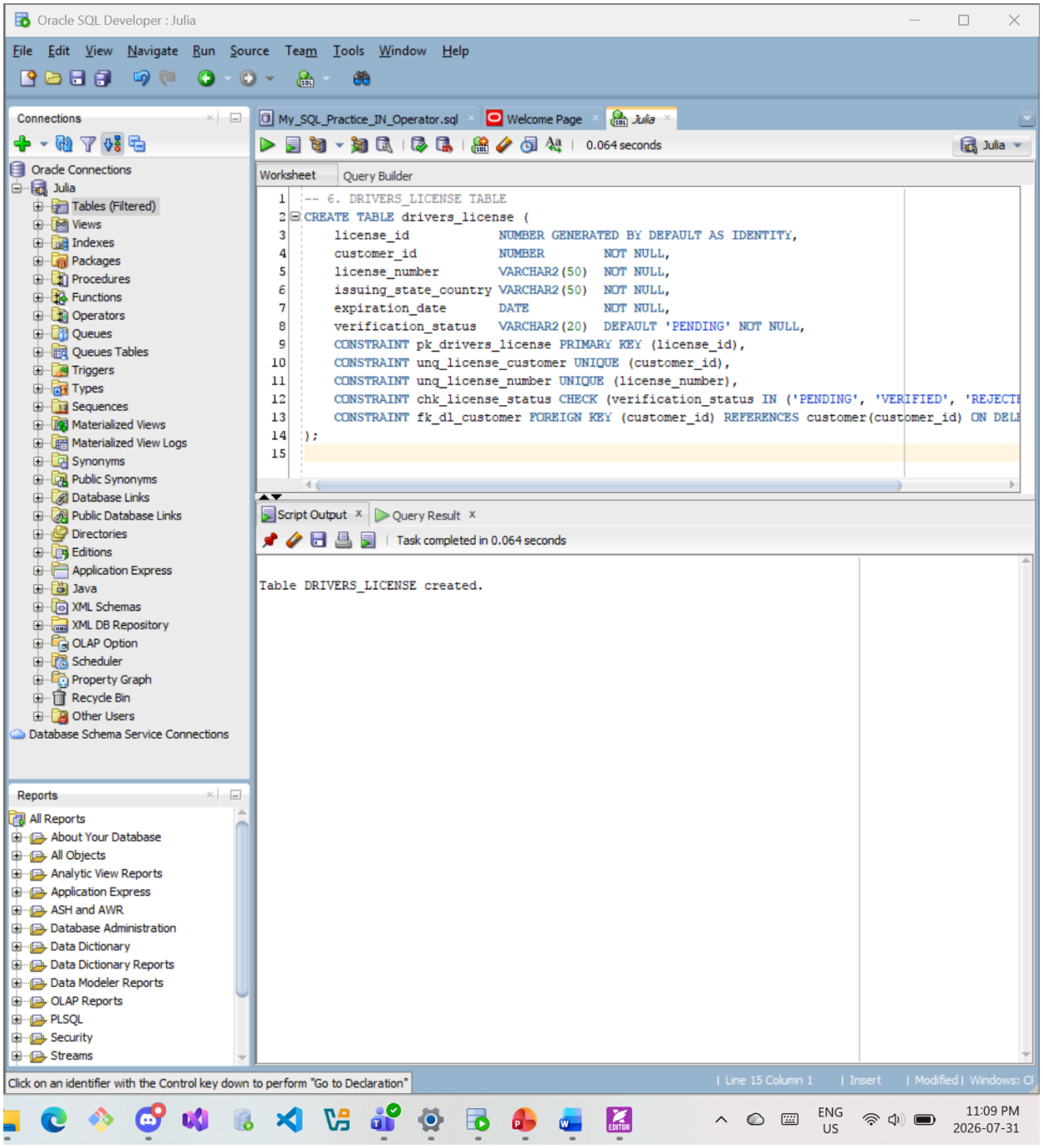


Table 7 – VEHICLE

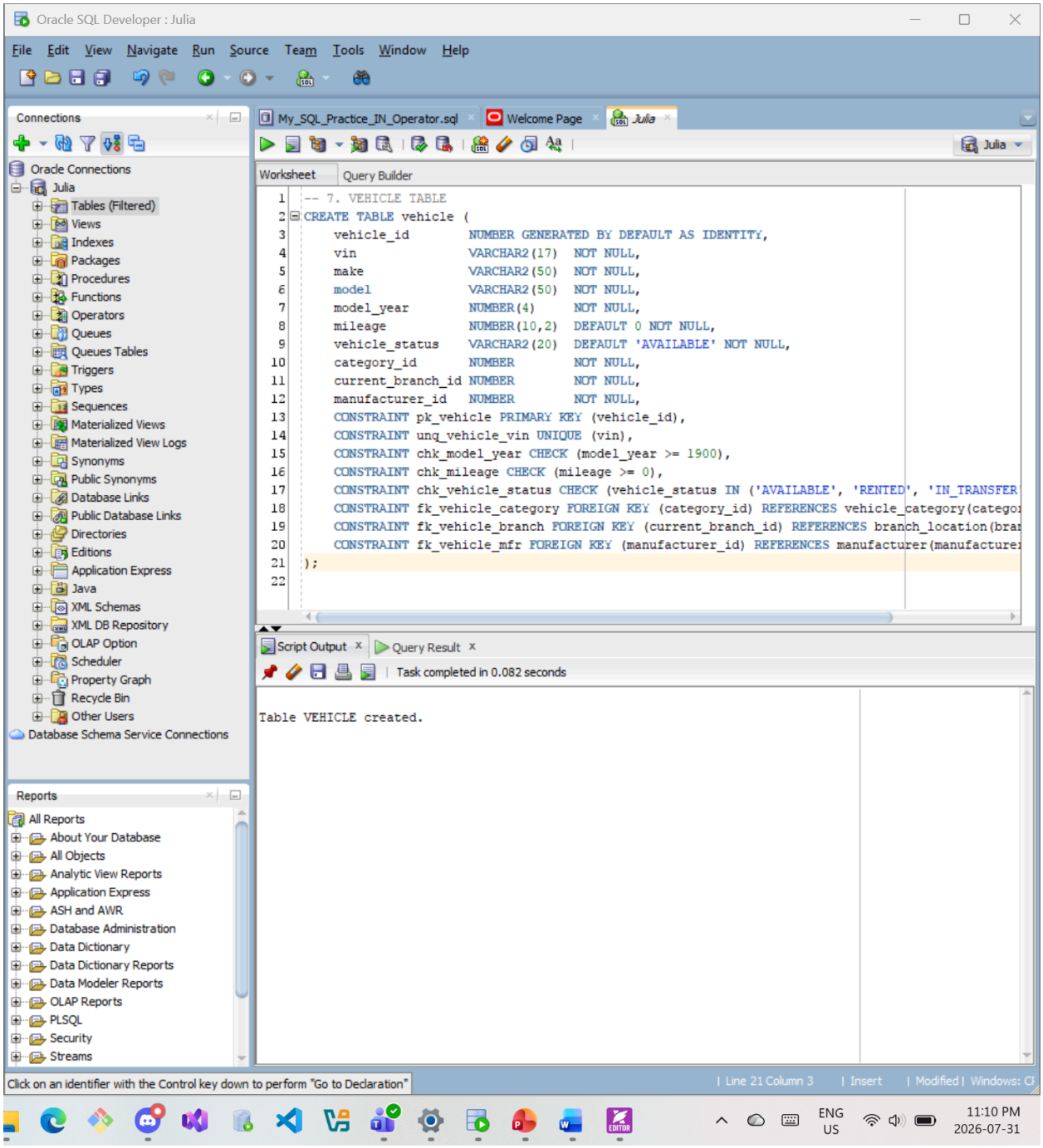


Table 8 – VEHICLE_WISHLIST

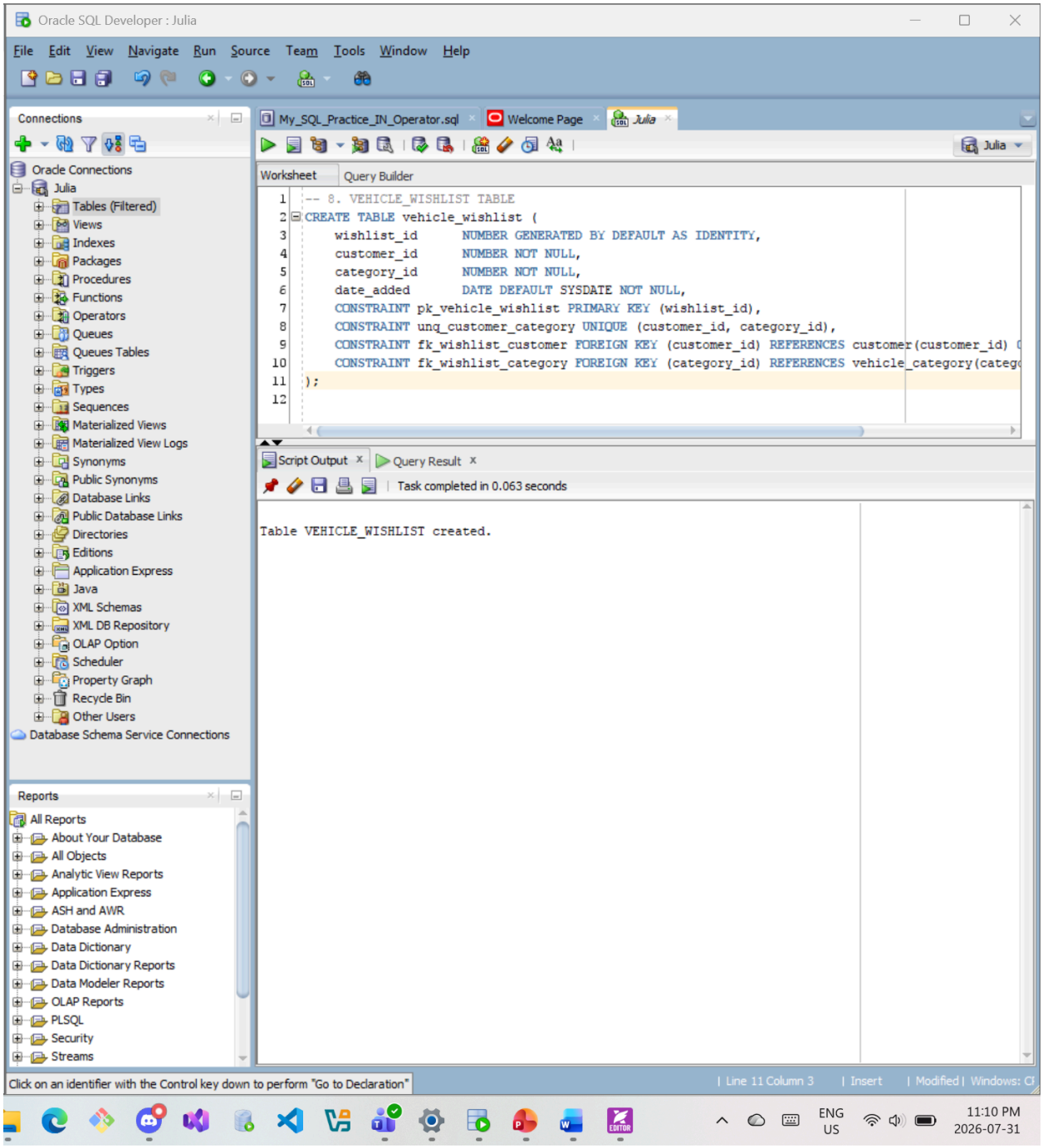


Table 9 – RESERVATION

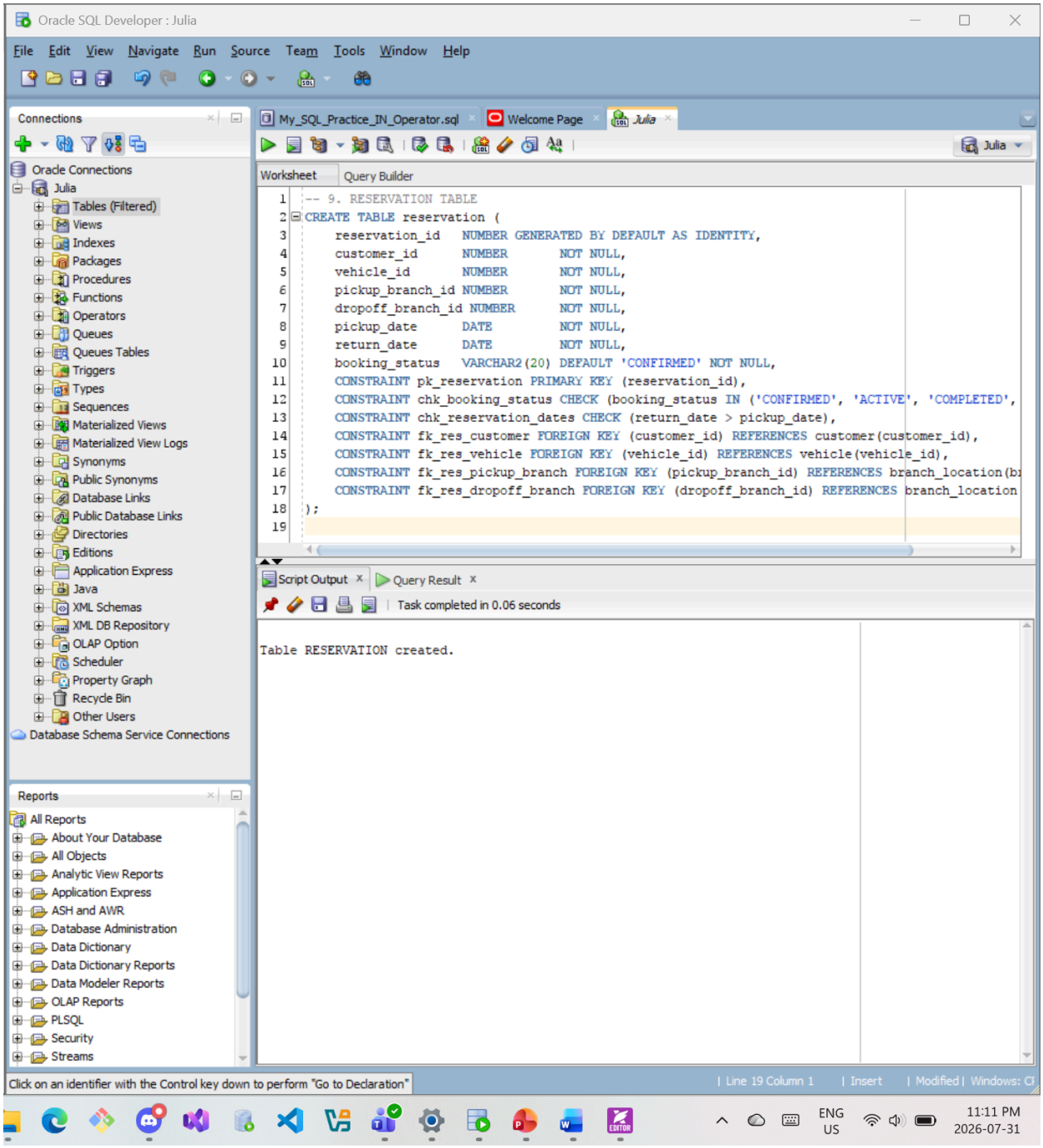


Table 10 – LOGISTICS_TRANSFER

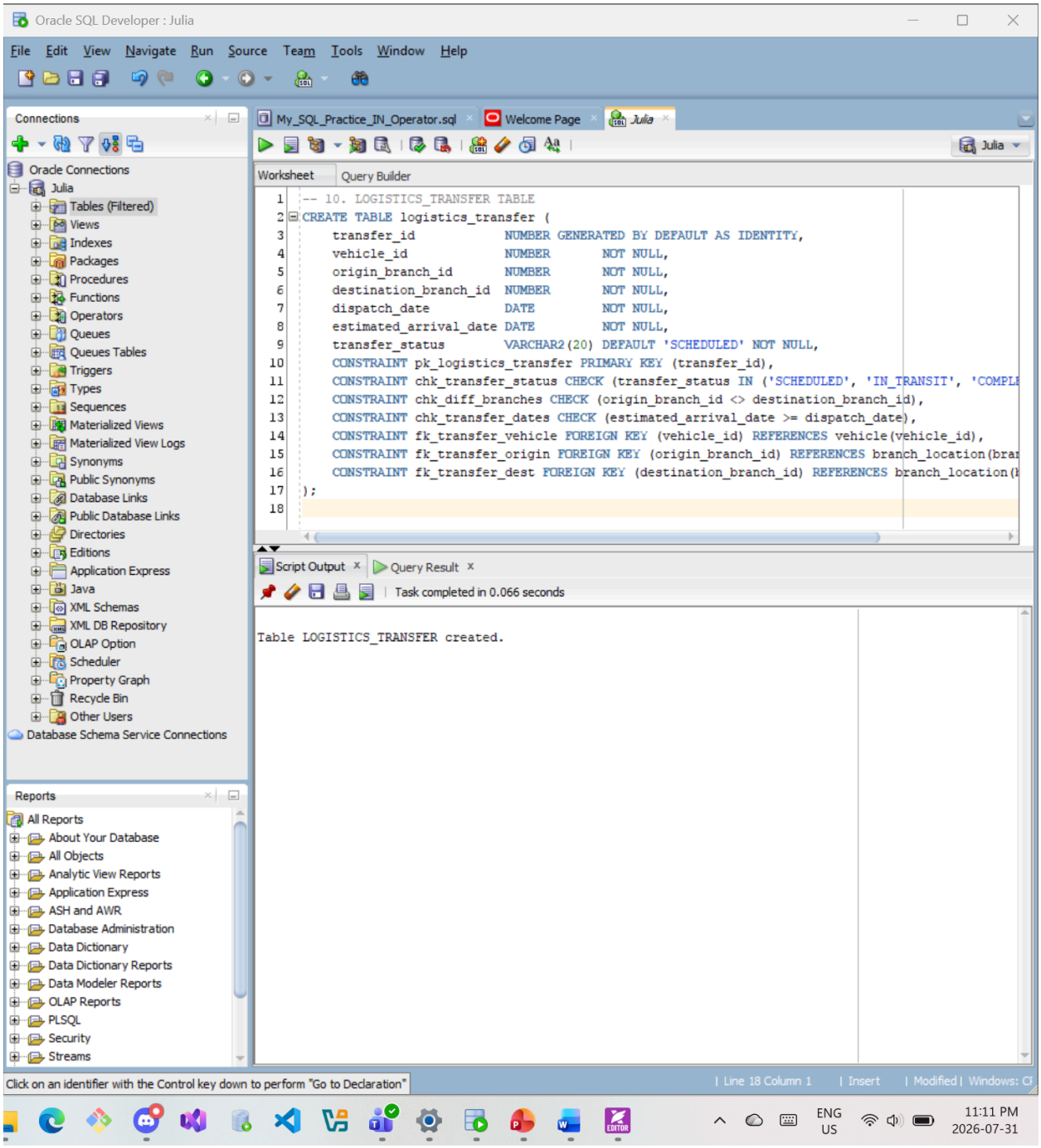


Table 11 – RENTAL_RETURN

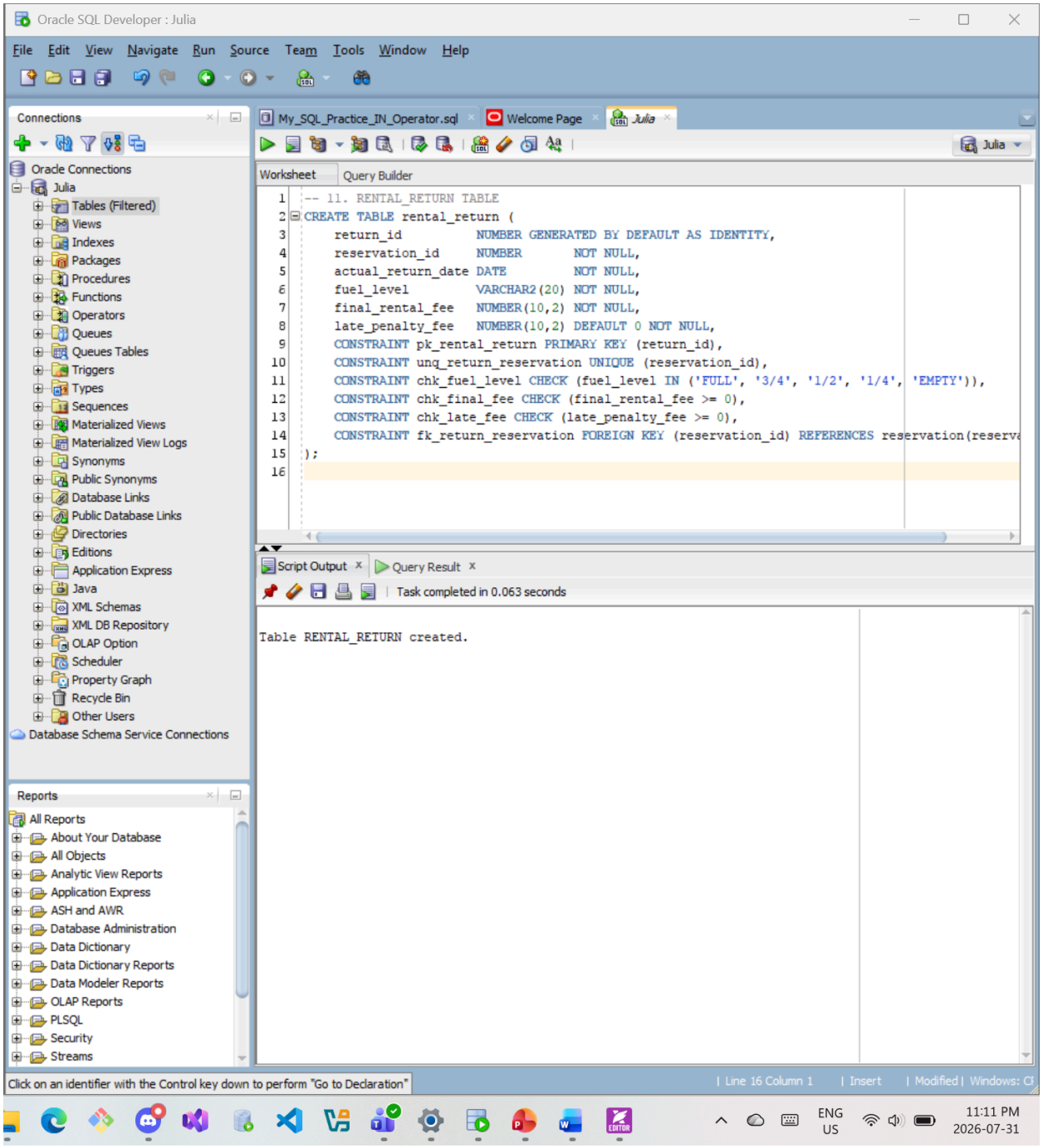
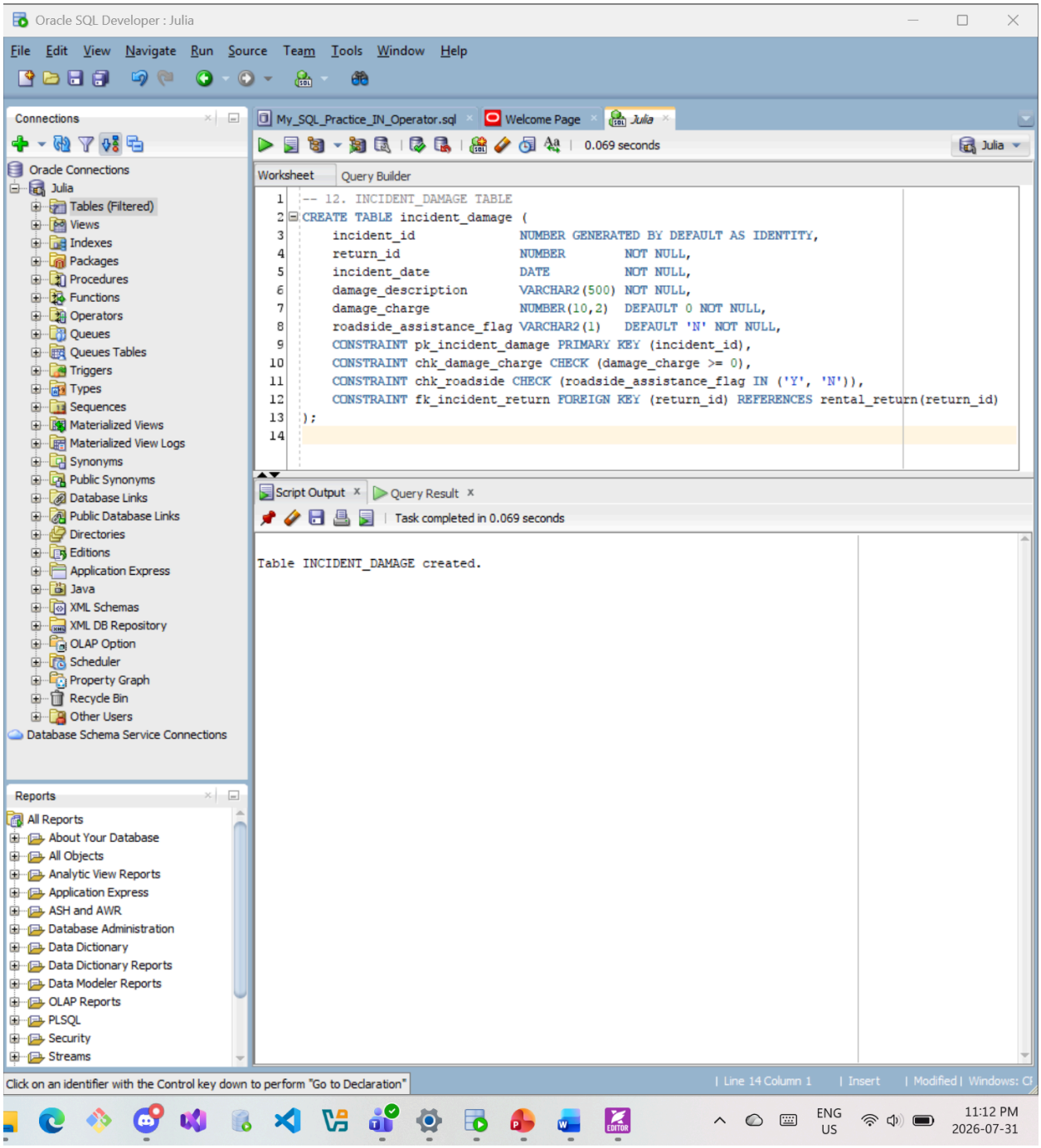


Table 12 – INCIDENT_DAMAGE



6.2.2 DML Script & Execution Log

Table 1 – BRANCH_LOCATION POPULATION

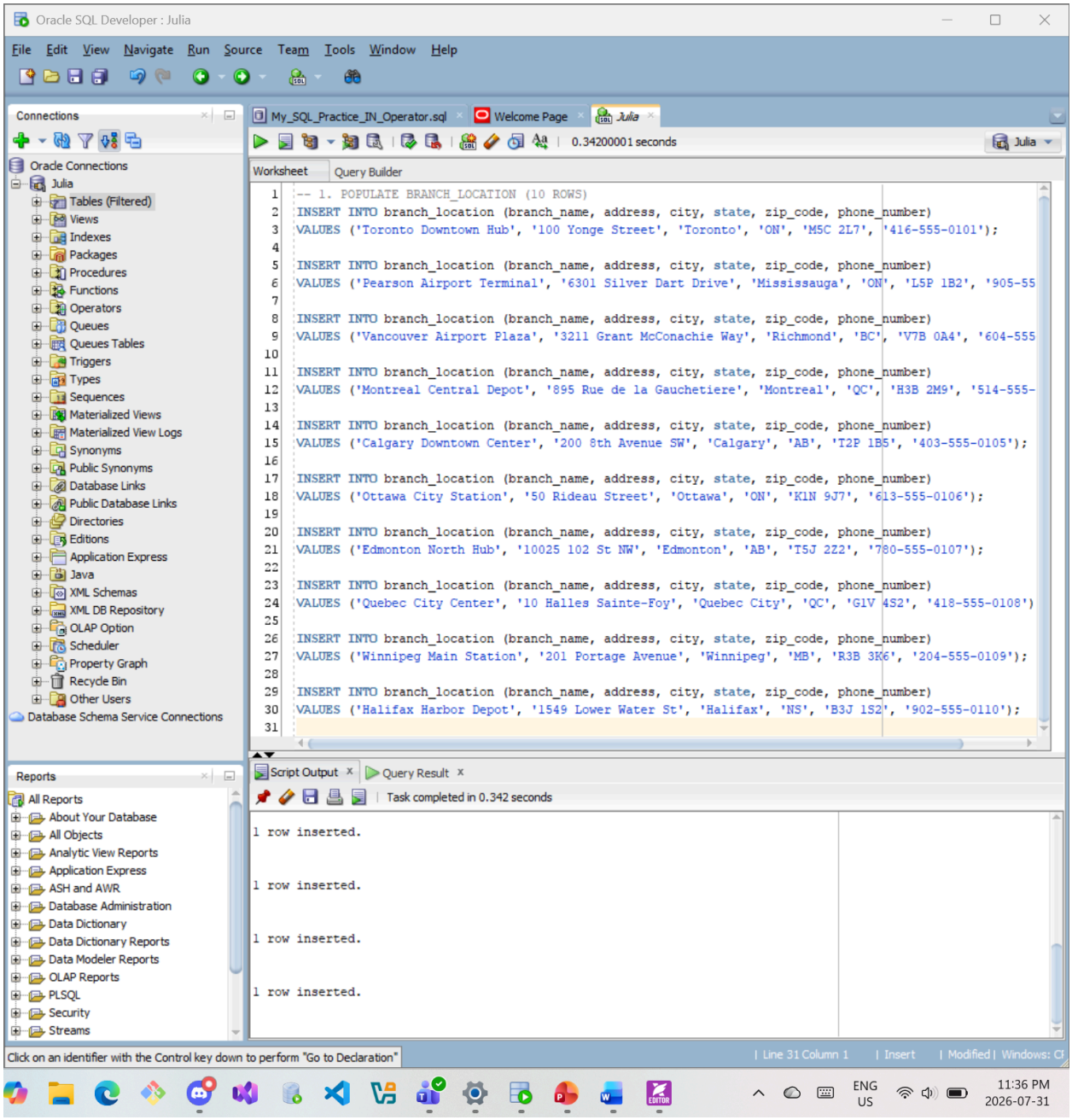


Table 2 – VEHICLE_CATEGORY POPULATION

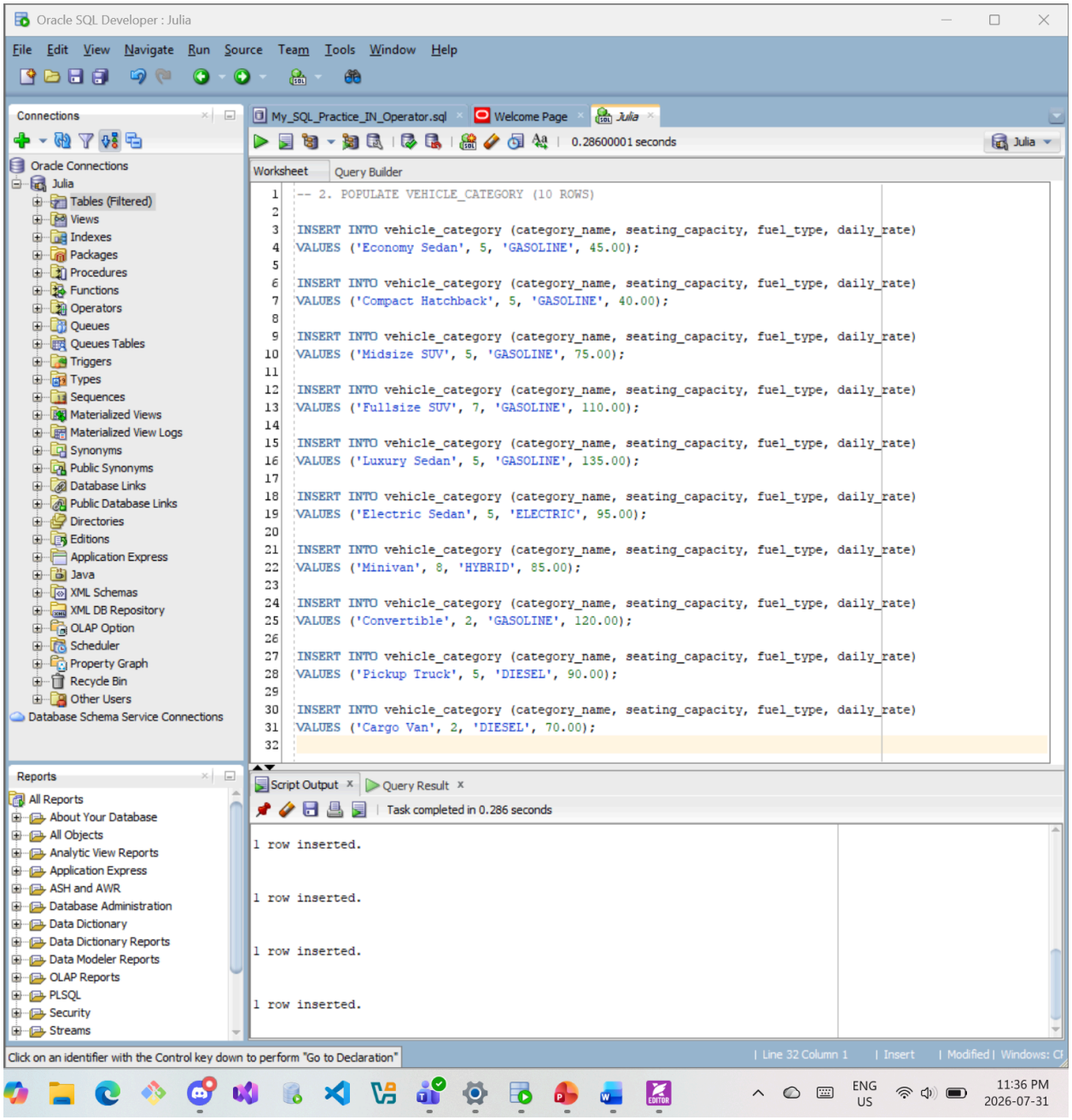


Table 3 – MANUFACTURER POPULATION

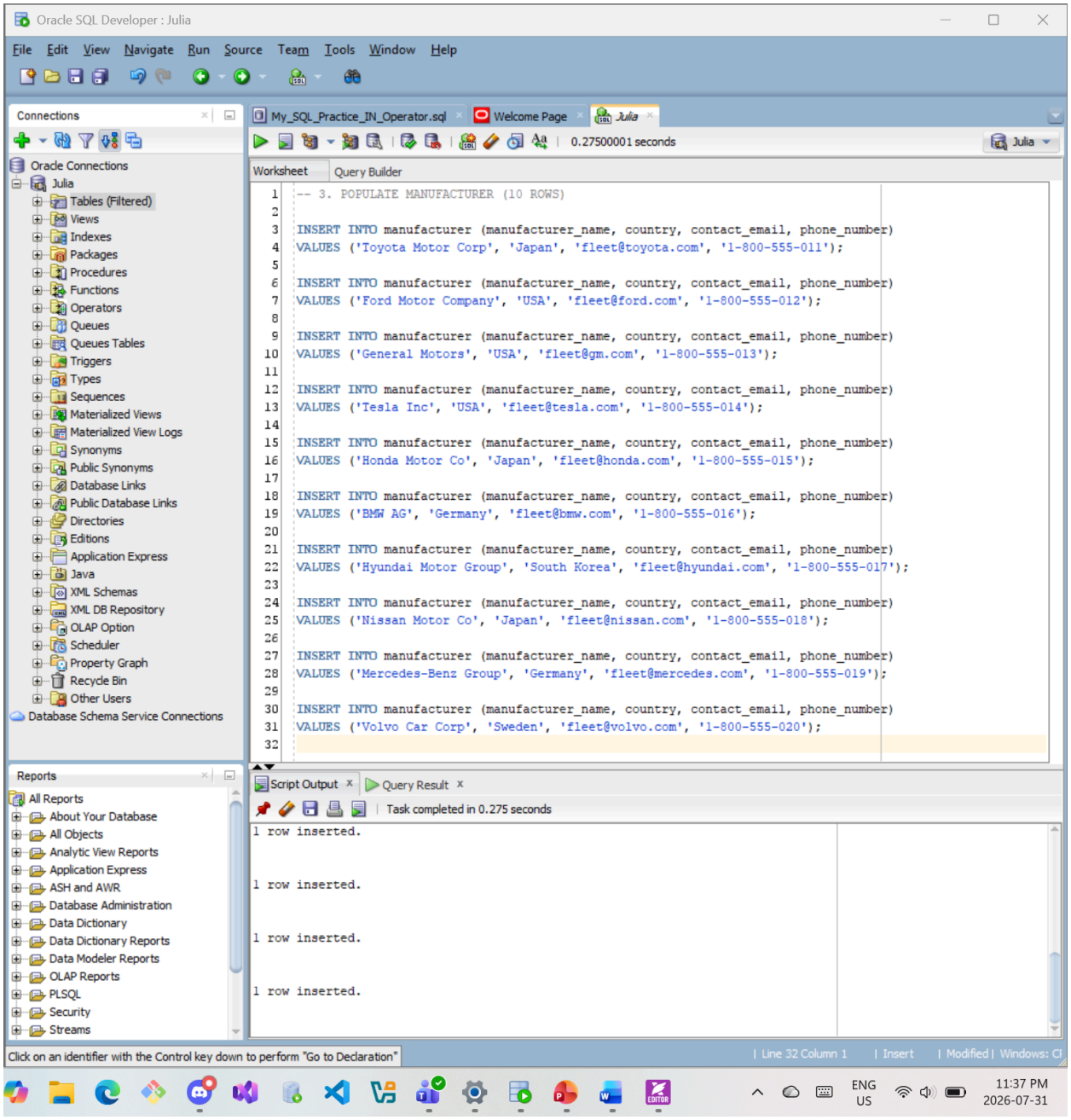


Table 4 – PARTS_VENDOR POPULATION

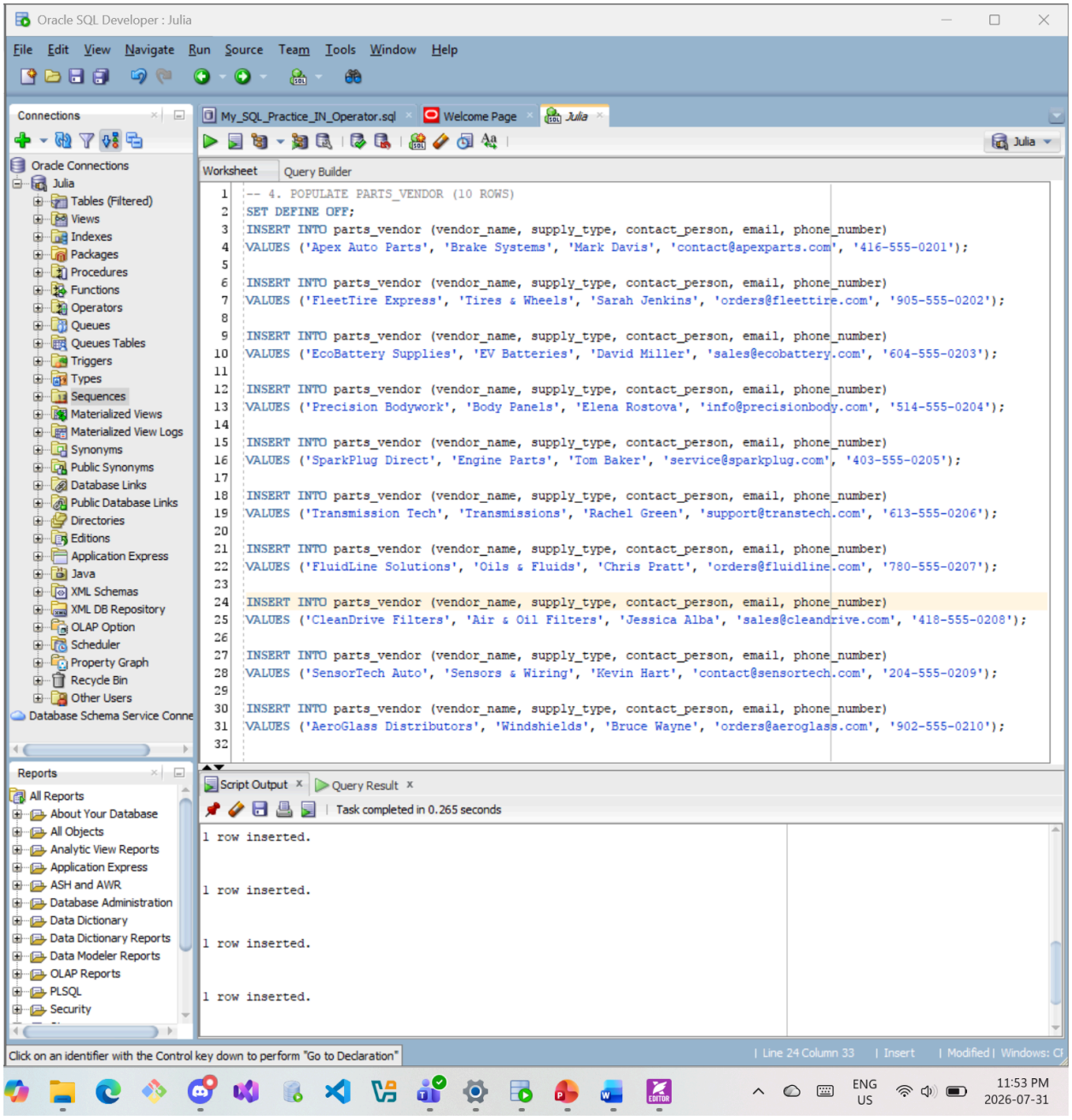


Table 5 – CUSTOMER POPULATION

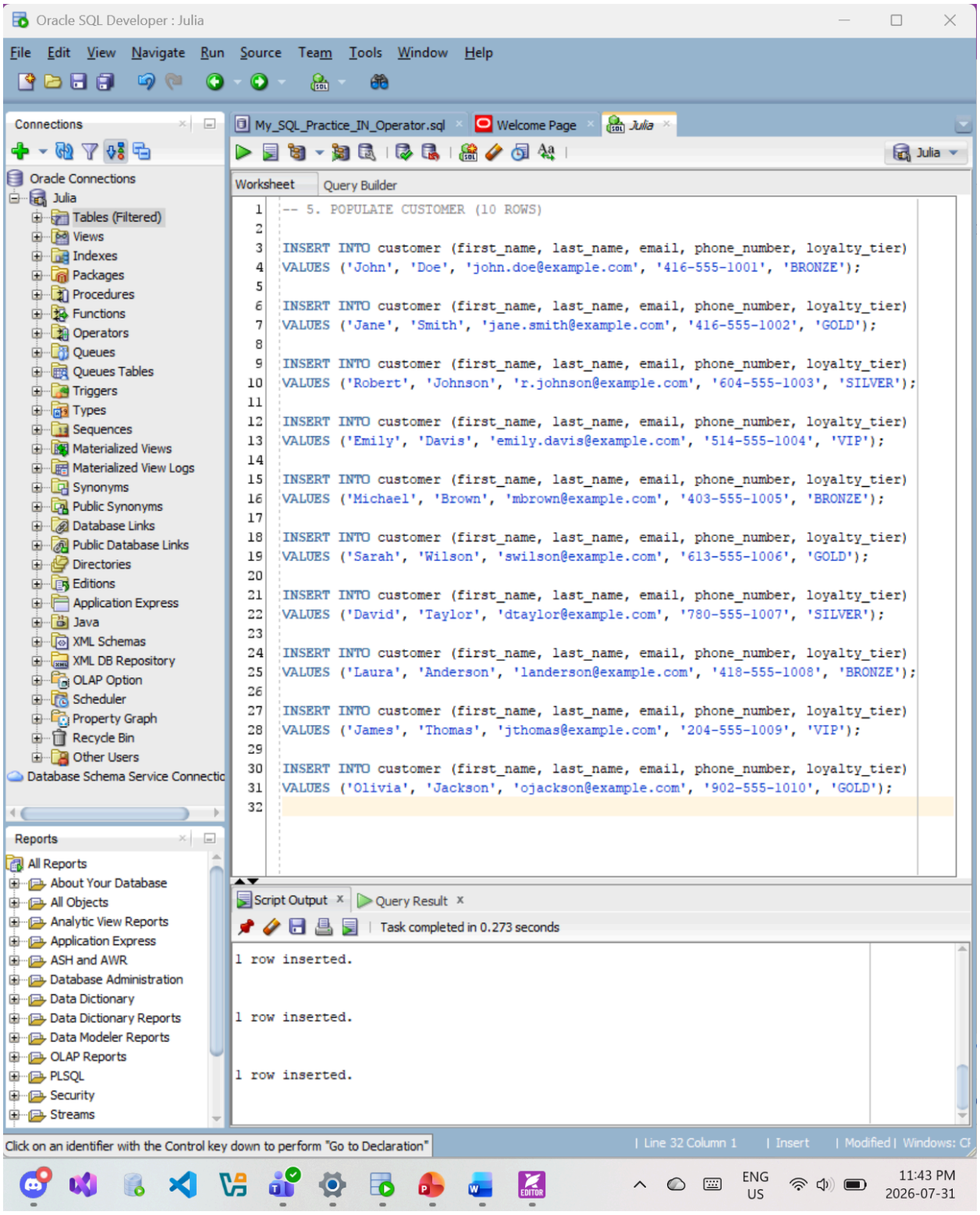


Table 6 – DRIVERS_LICENSE POPULATION

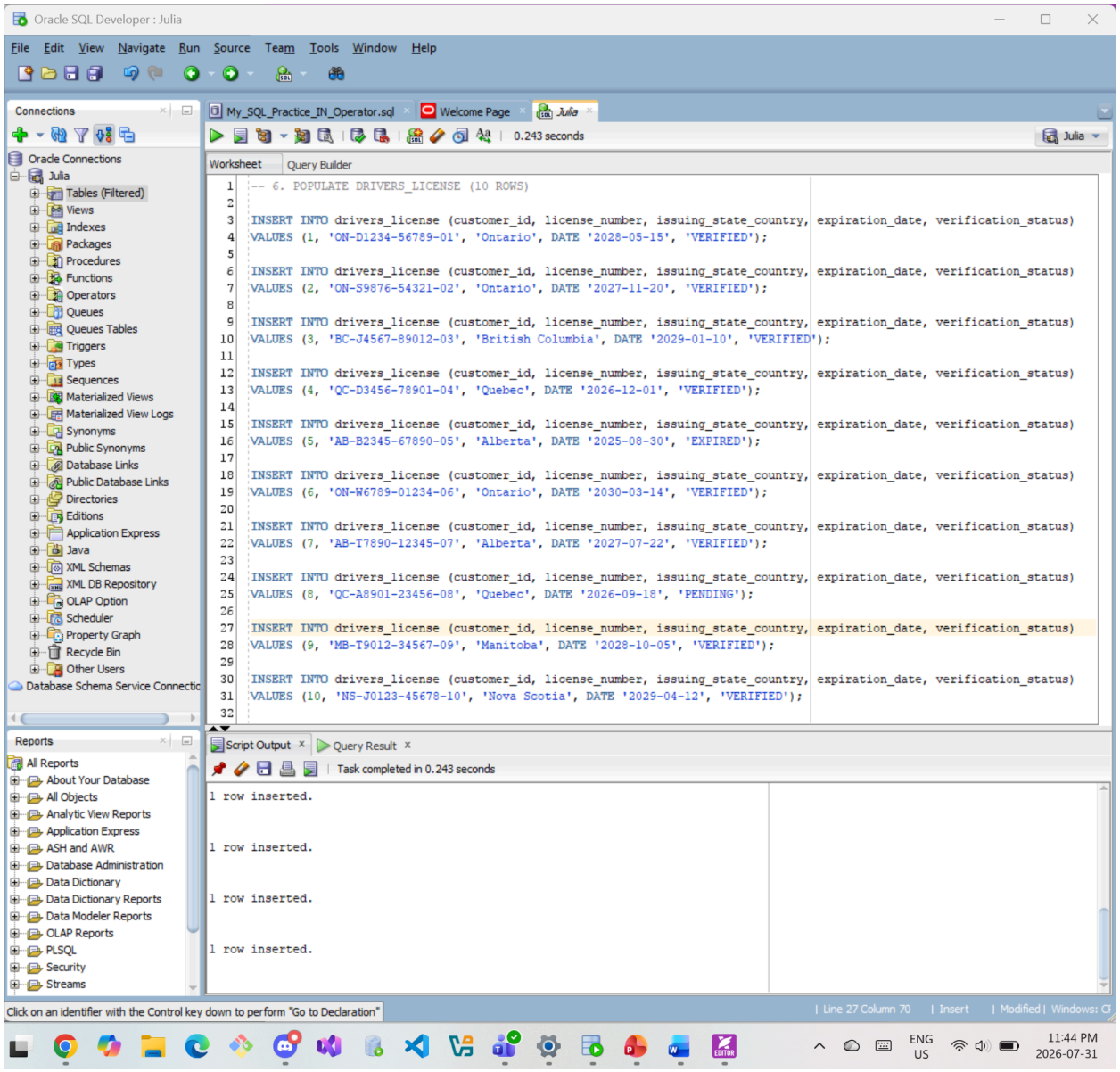


Table 7 – VEHICLE POPULATION

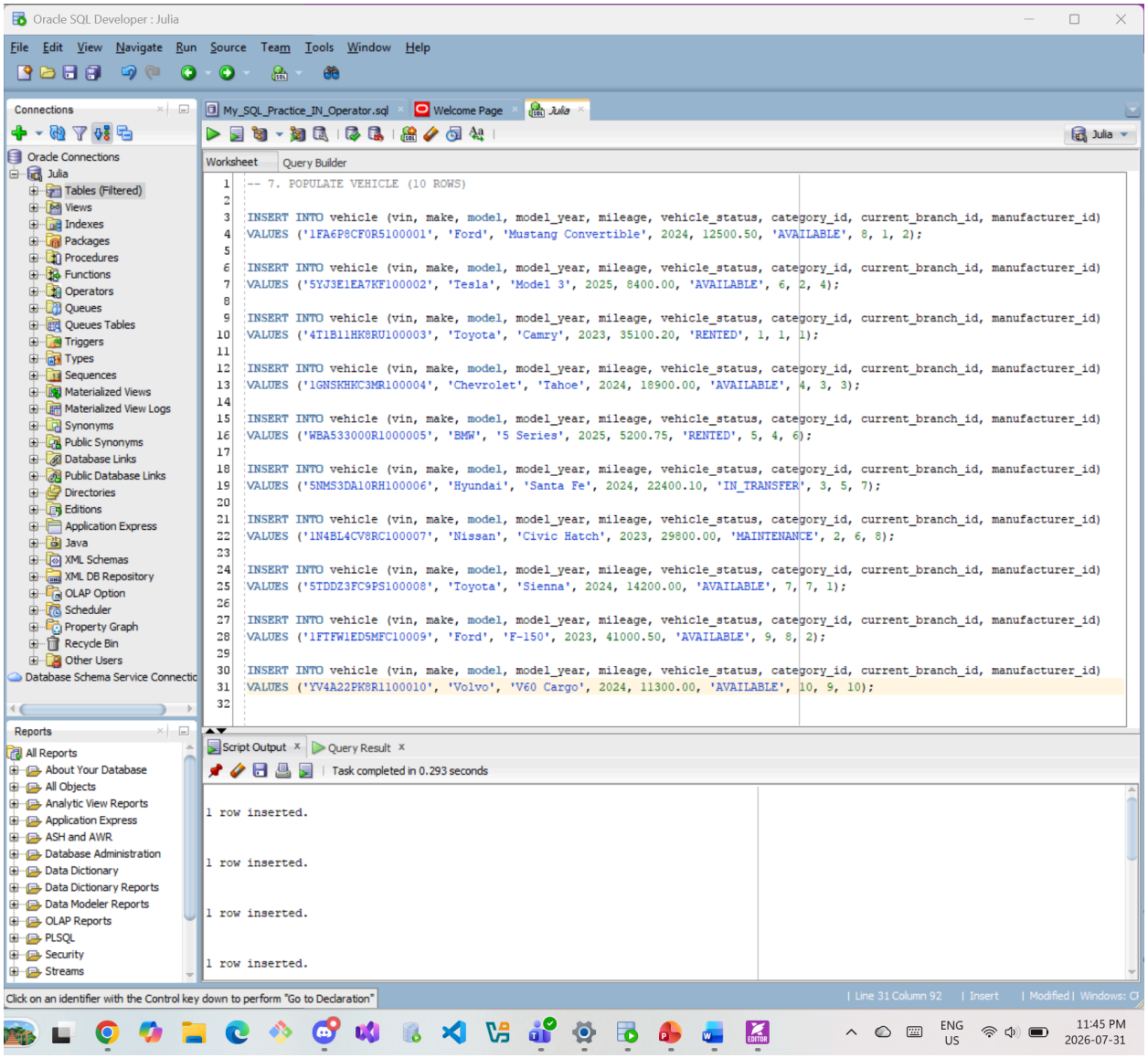


Table 8 – VEHICLE_WISHLIST POPULATION

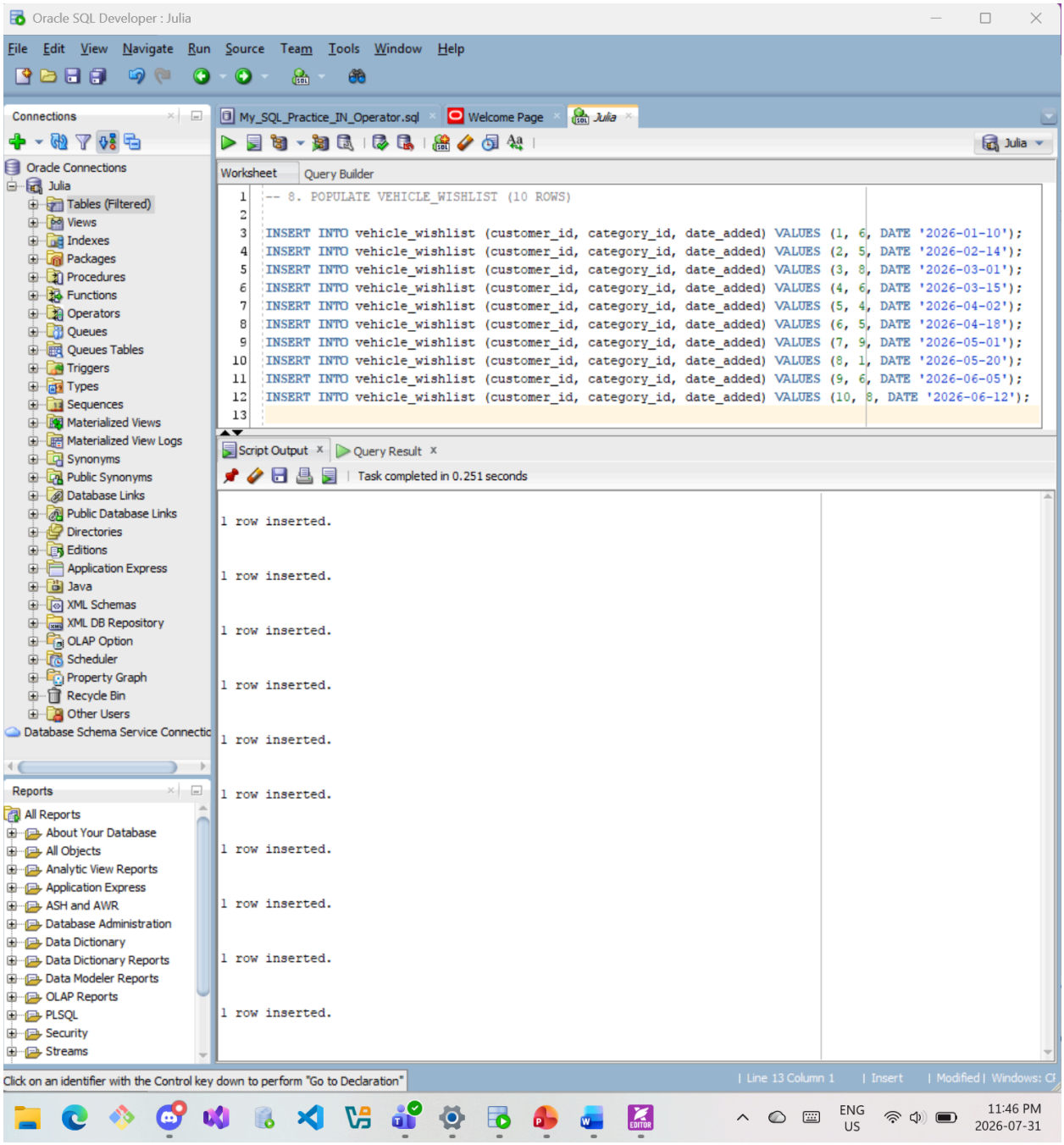


Table 9 – RESERVATION POPULATION

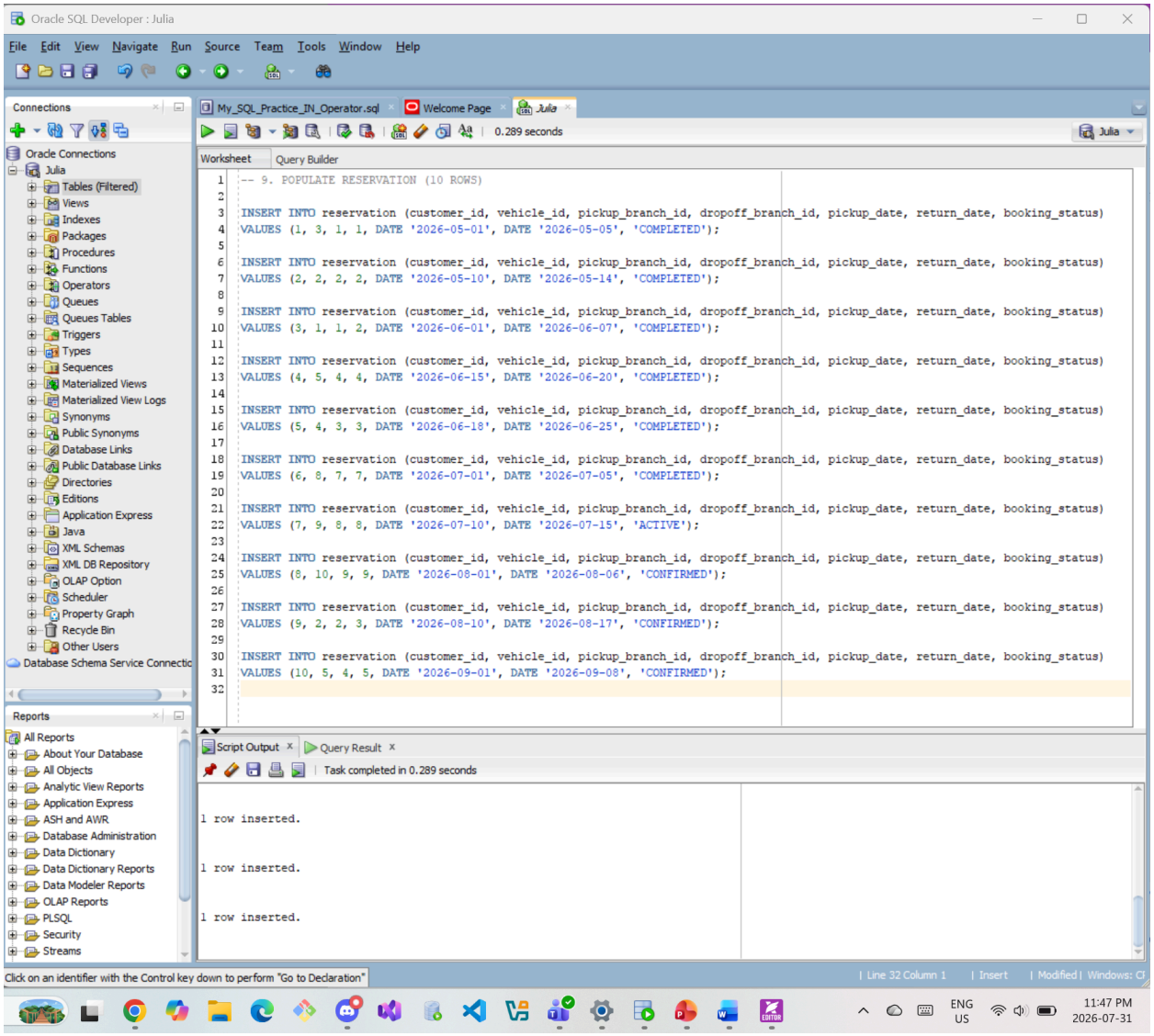


Table 10 – LOGISTICS_TRANSFER POPULATION

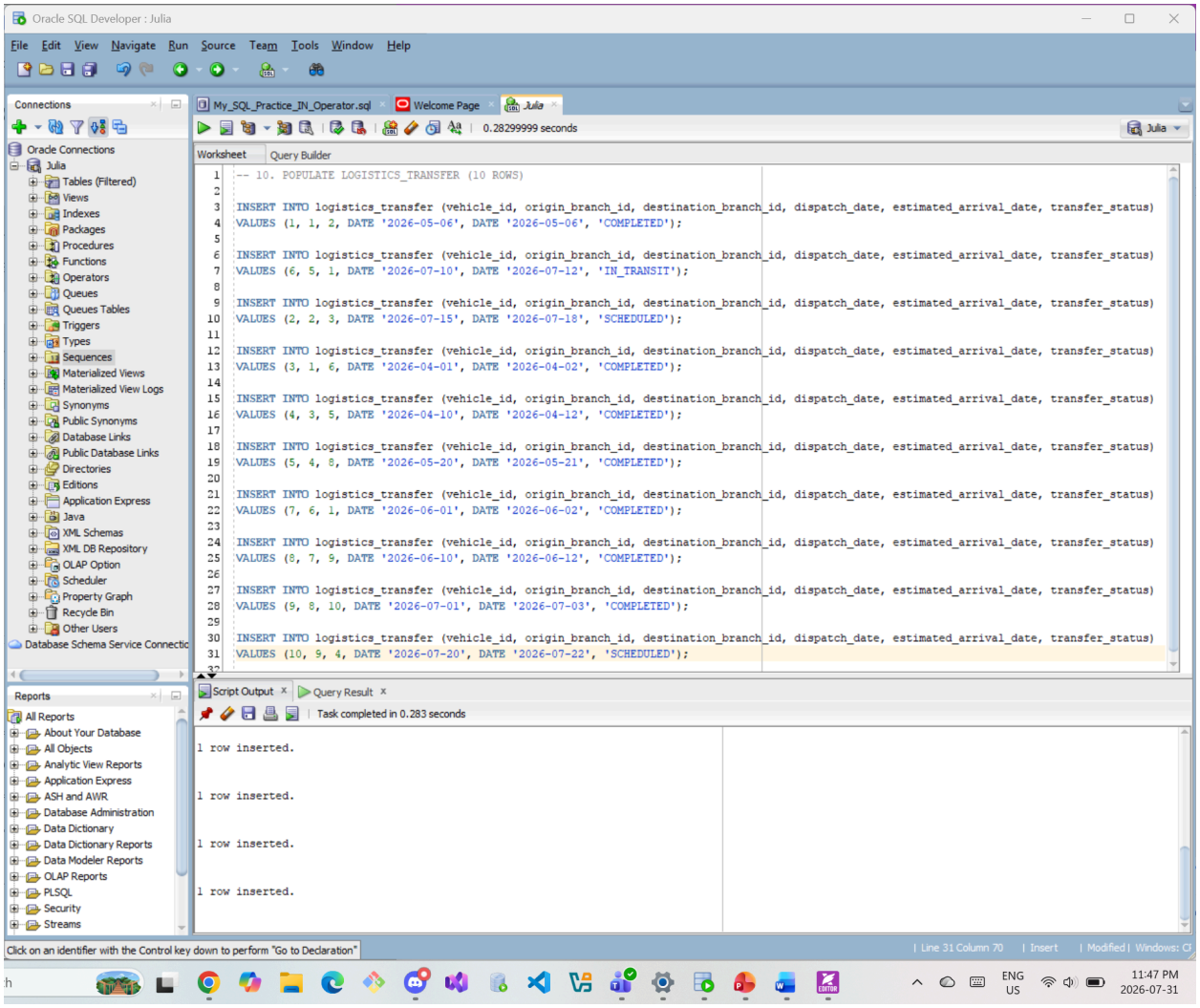


Table 11 – RENTAL_RETURN POPULATION

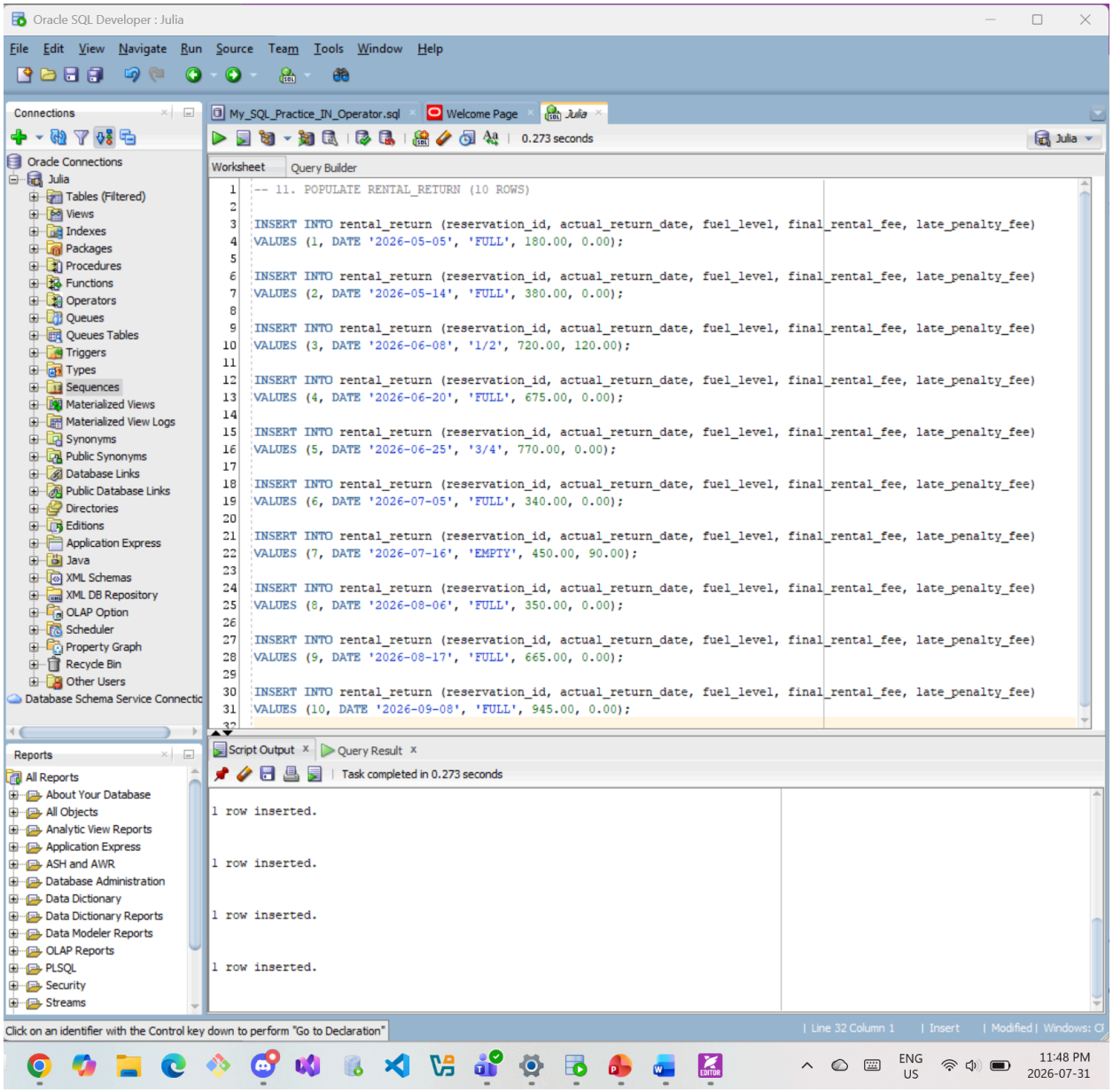
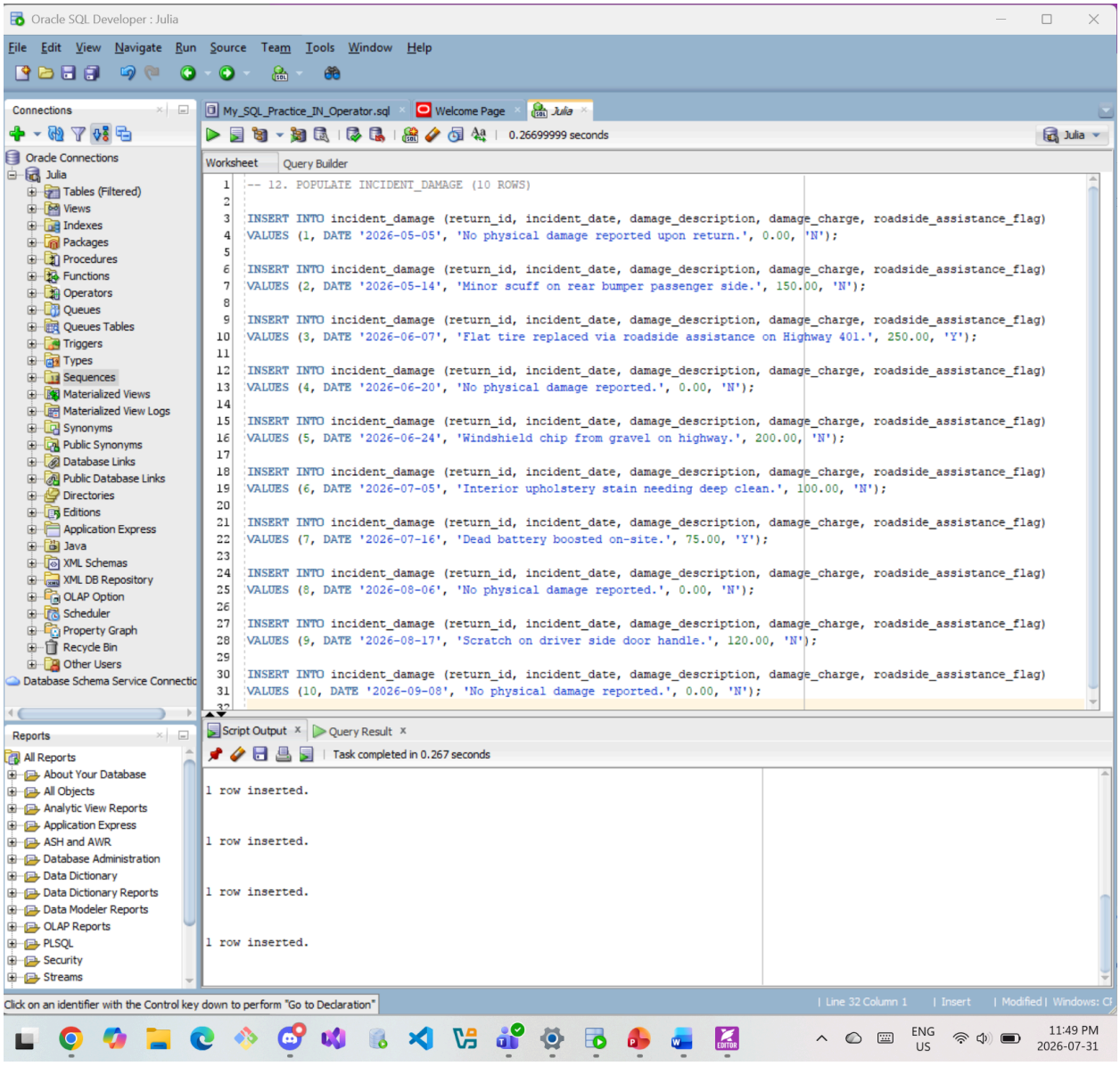
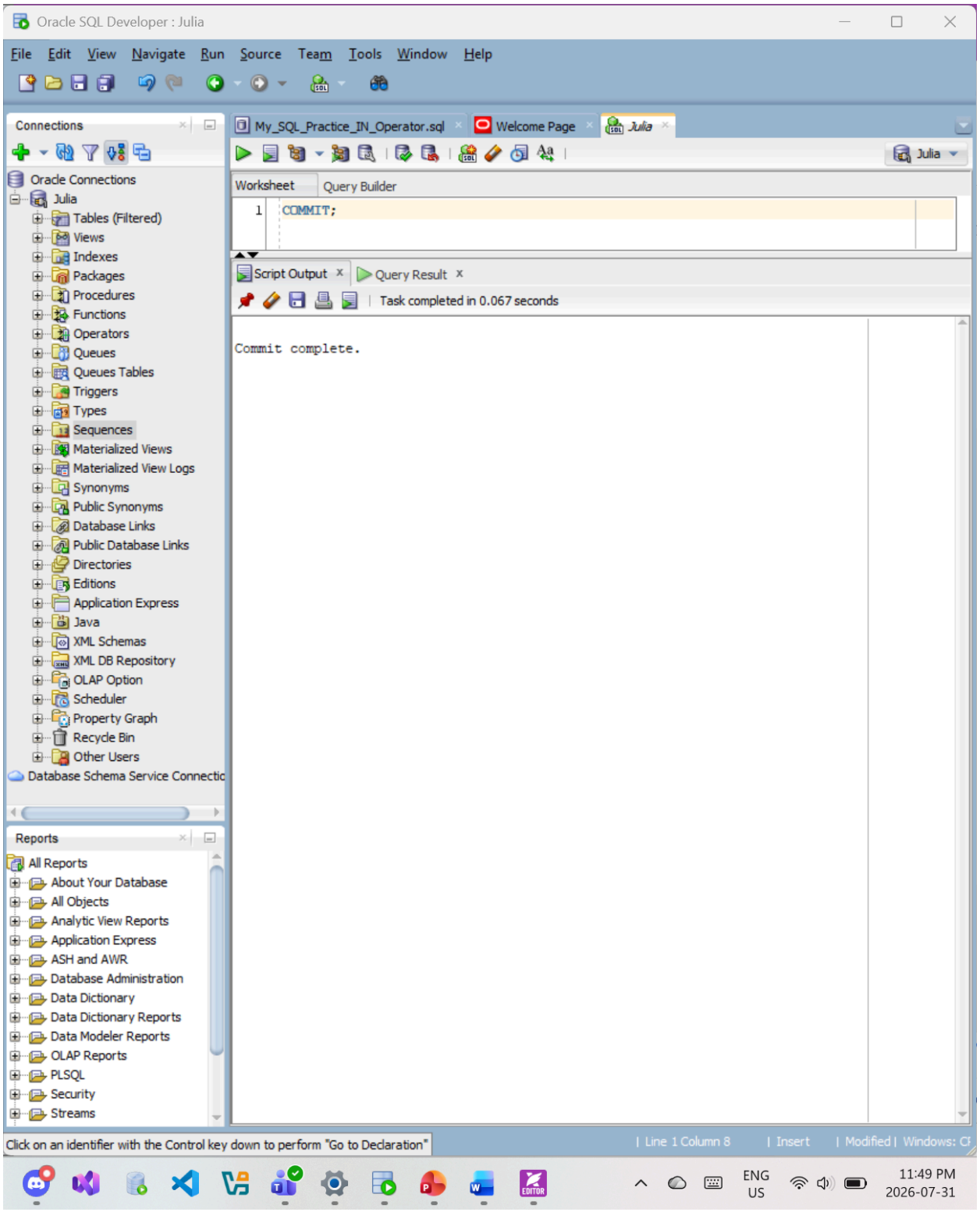


Table 12 – INCIDENT_DAMAGE POPULATION



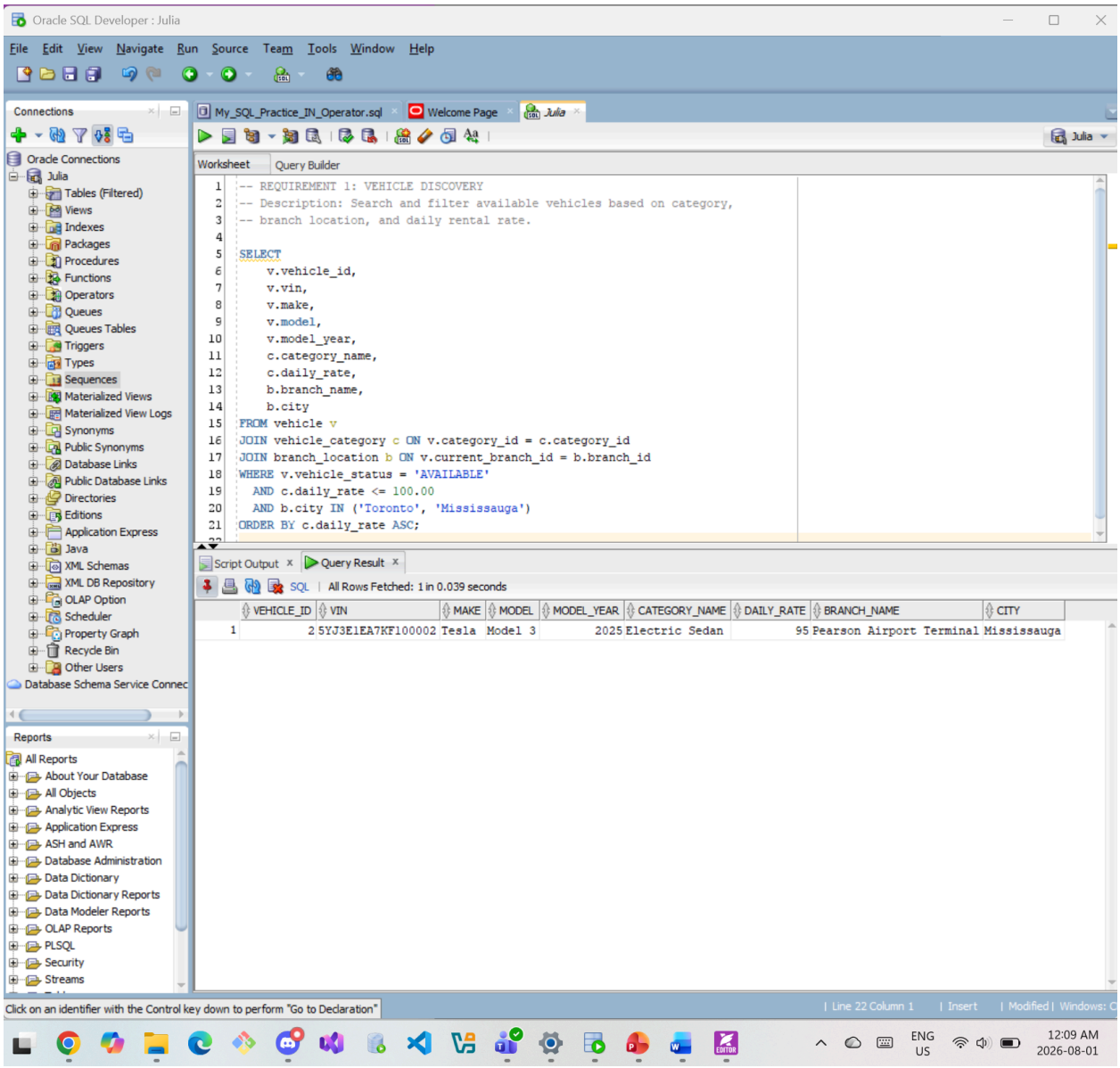
COMMIT ALL TRANSACTION CHANGES TO ORACLE



6.2.3 Core Business Requirement Queries & Execution Log

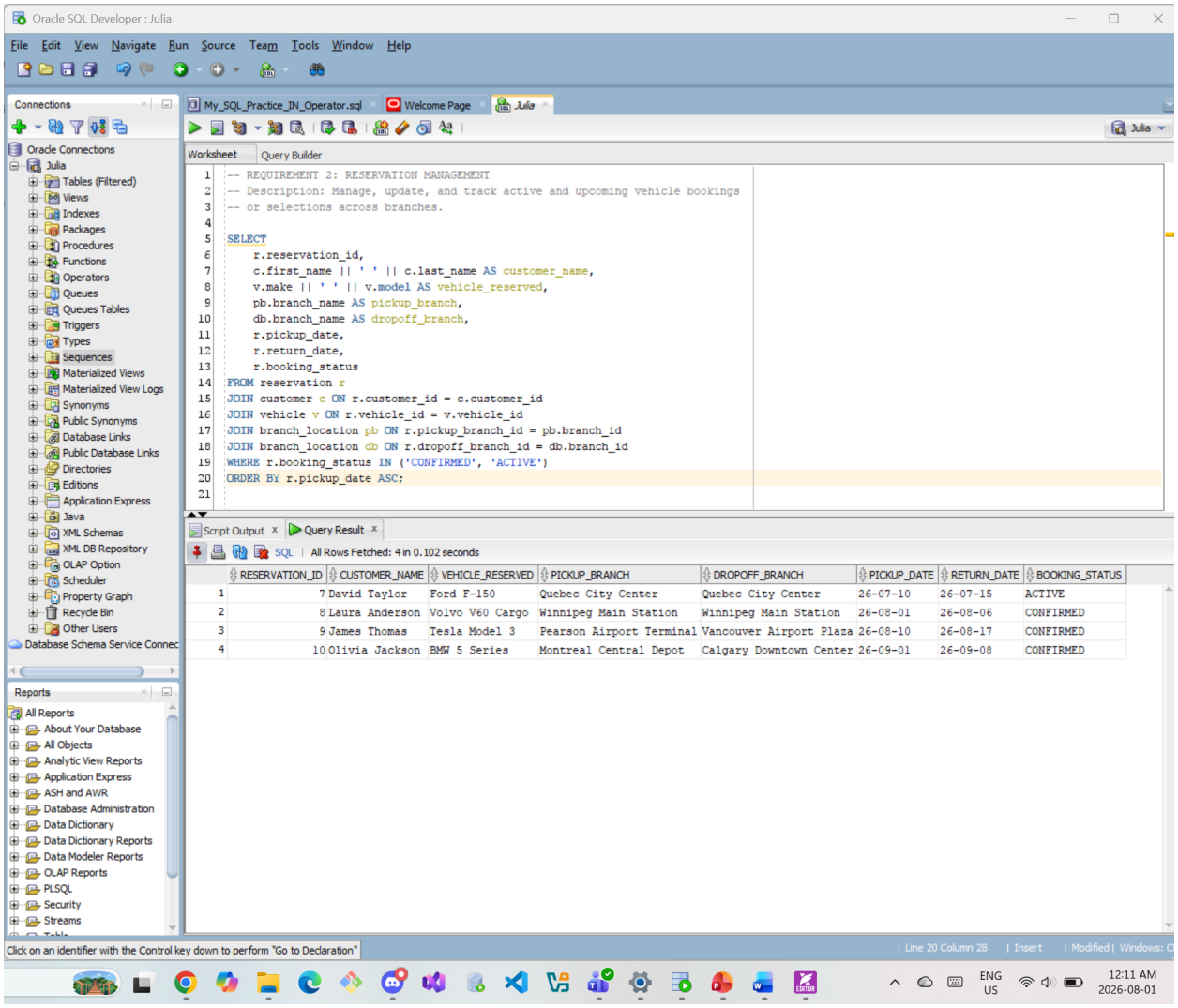
#1 VEHICLE DISCOVERY

Description: Search and filter available vehicles based on category, branch location, and daily rental rate.



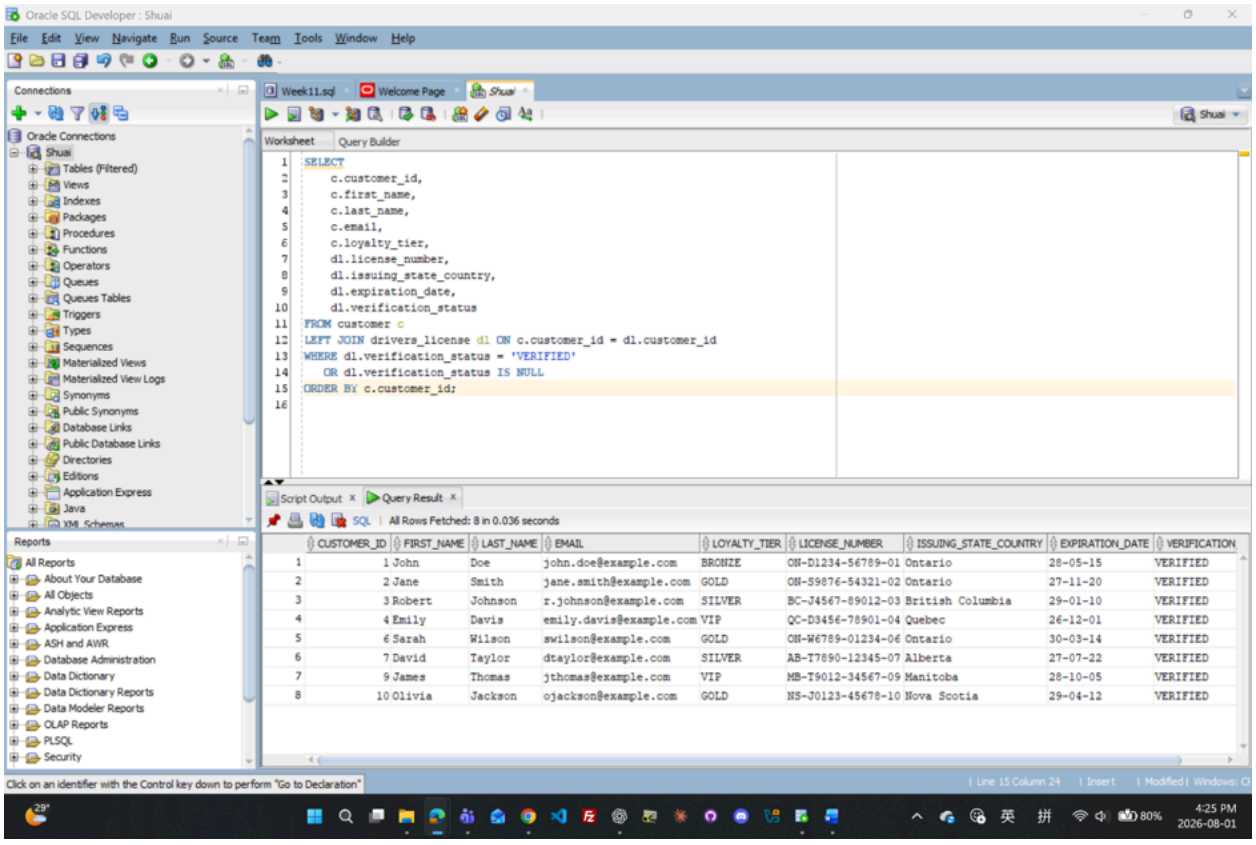
#2 RESERVATION MANAGEMENT

Description: Manage, update, and track active and upcoming vehicle bookings or selections across branches.



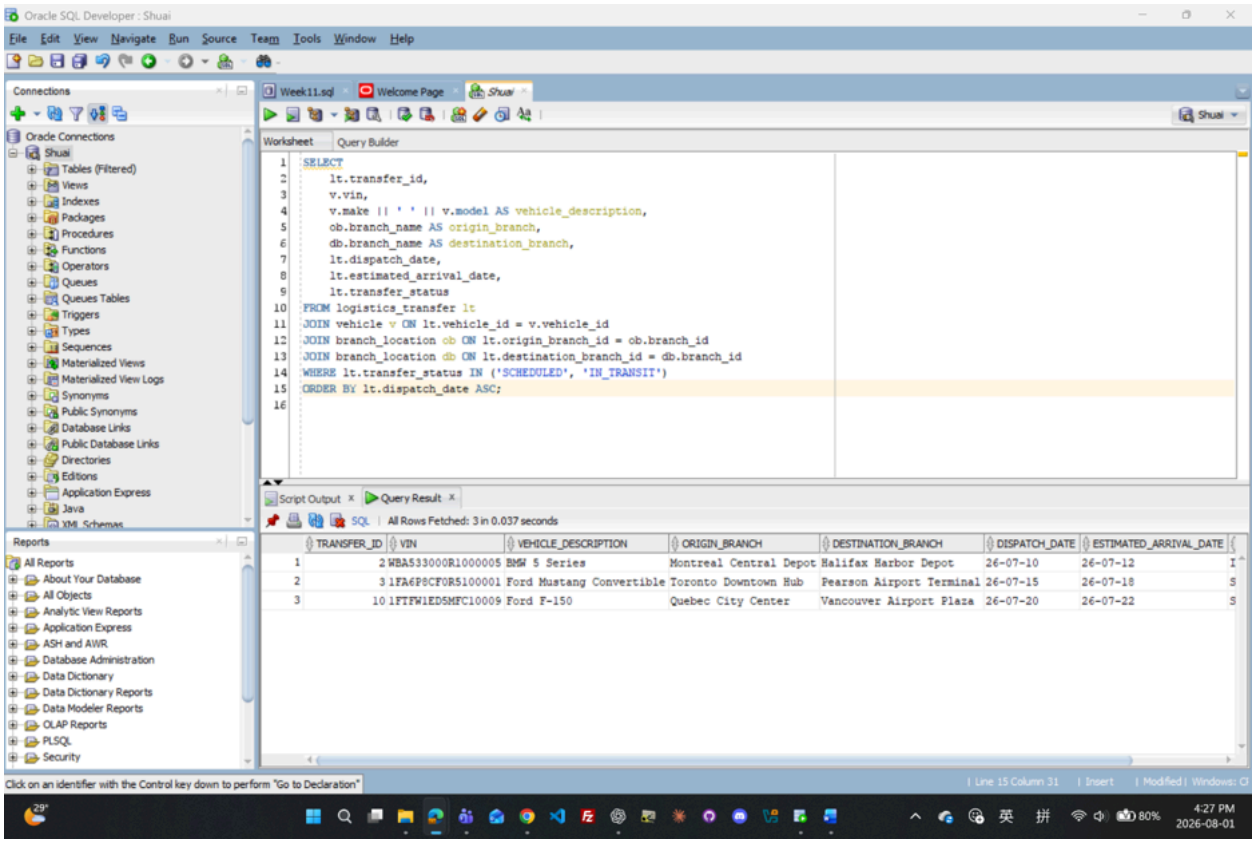
#3 USER ACCOUNT MANAGEMENT

Description: Handle customer profiles, driver's license verifications, and account status tracking.



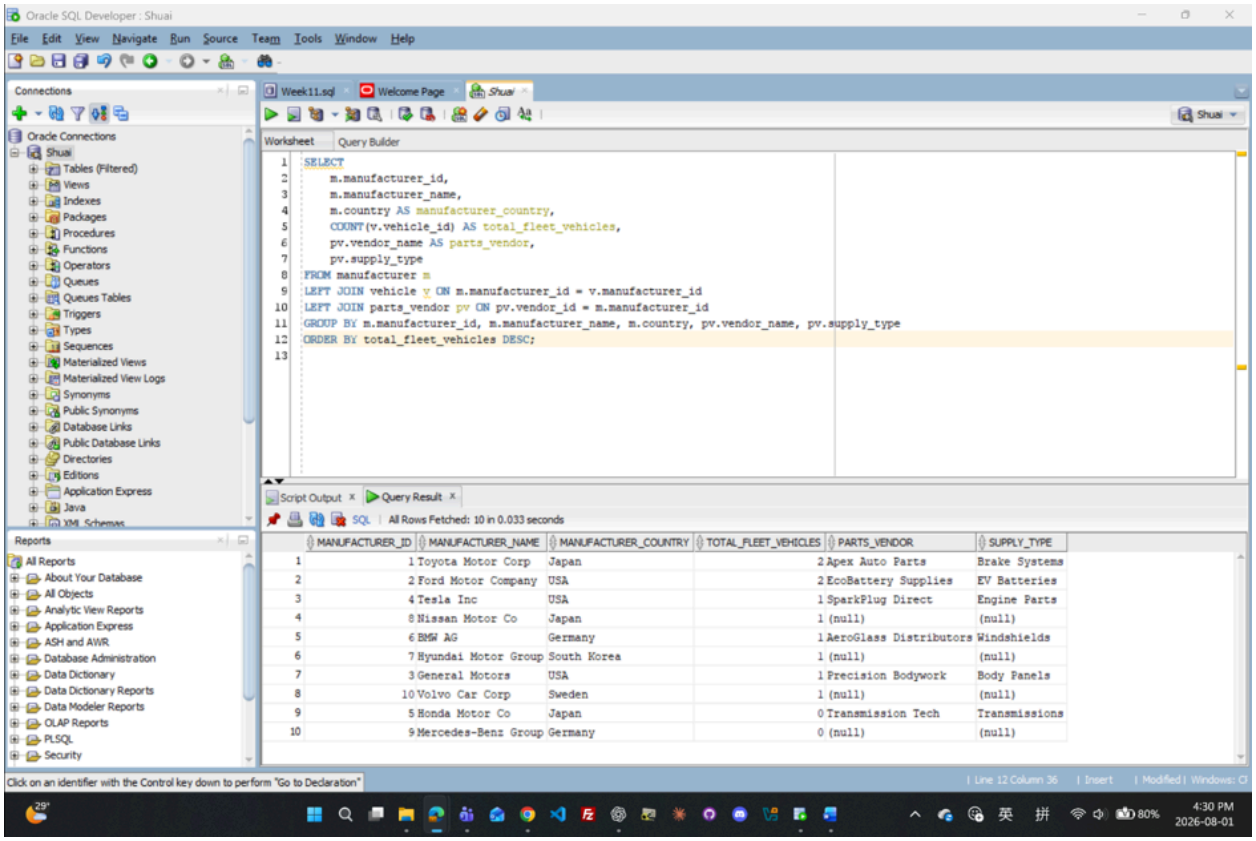
#4 LOGISTICS TRACKING

Description: Track operational shipping status, transfers, and fulfillment status of vehicles moving between branches.



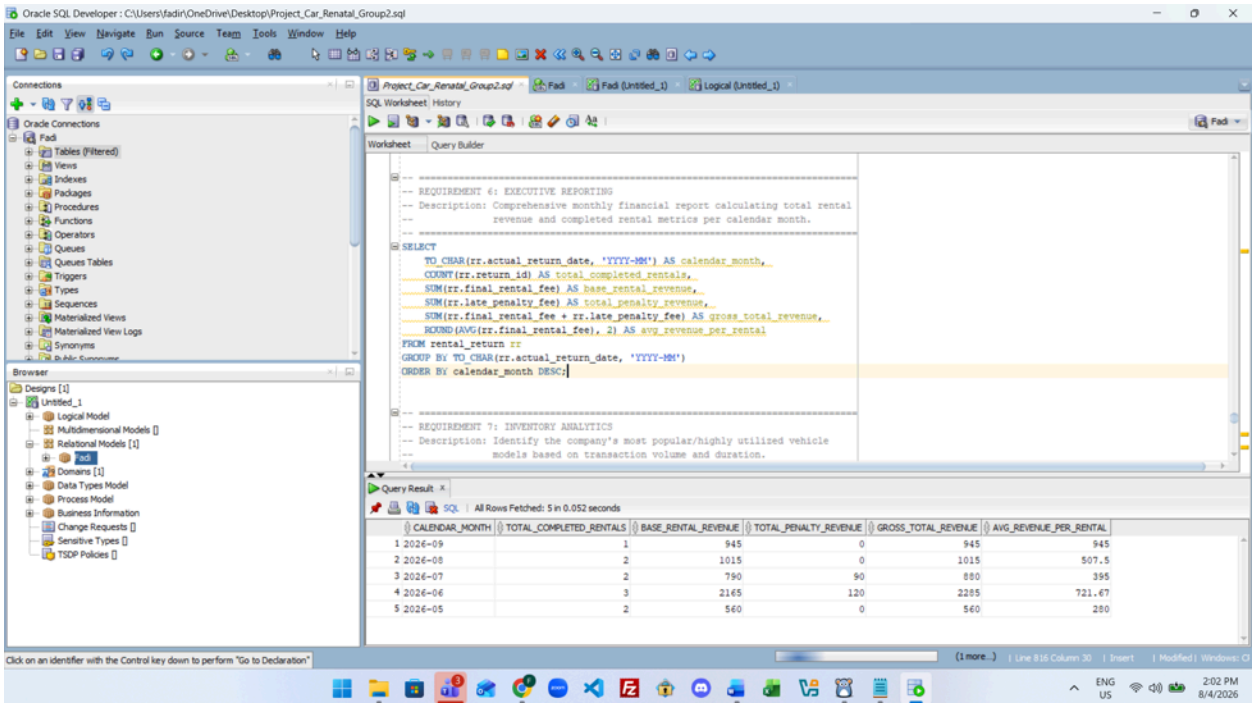
#5 SUPPLY CHAIN MANAGEMENT

Description: Manage vehicle manufacturers, external parts vendors, and third-party procurement mapping.



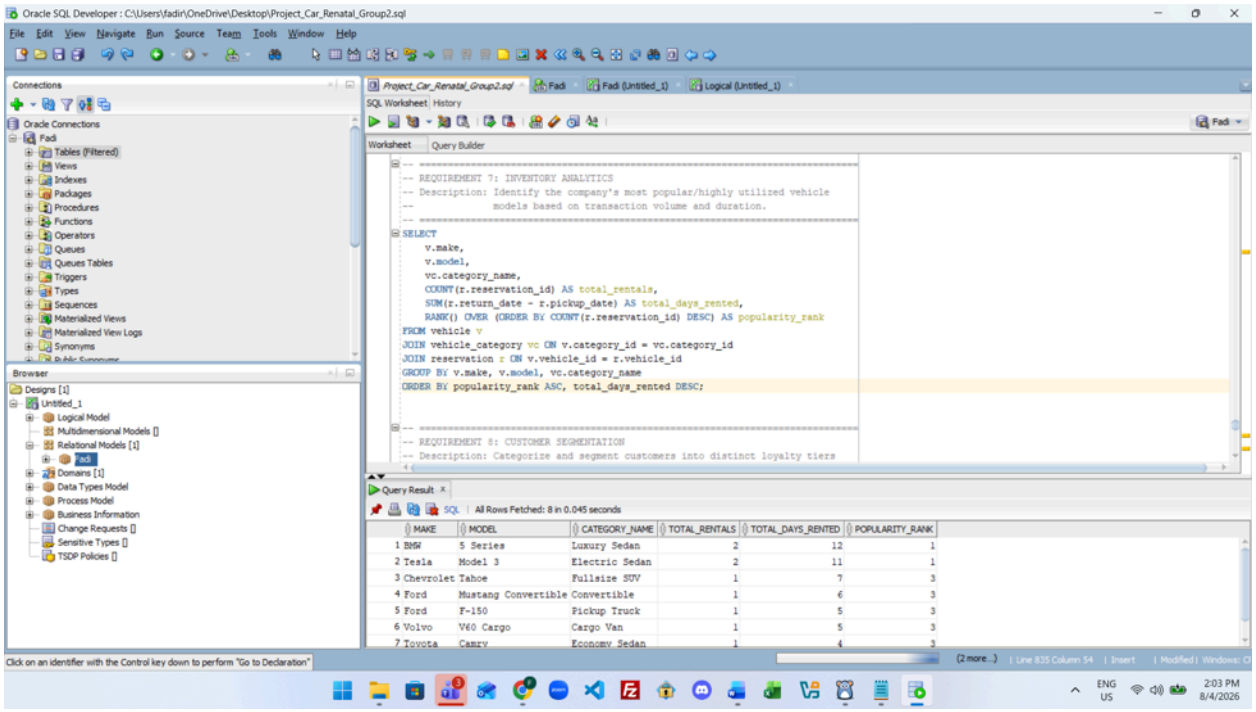
#6 EXECUTIVE REPORTING

Description: Generate comprehensive monthly financial report calculating total rental revenue and utilization metrics per calendar month.



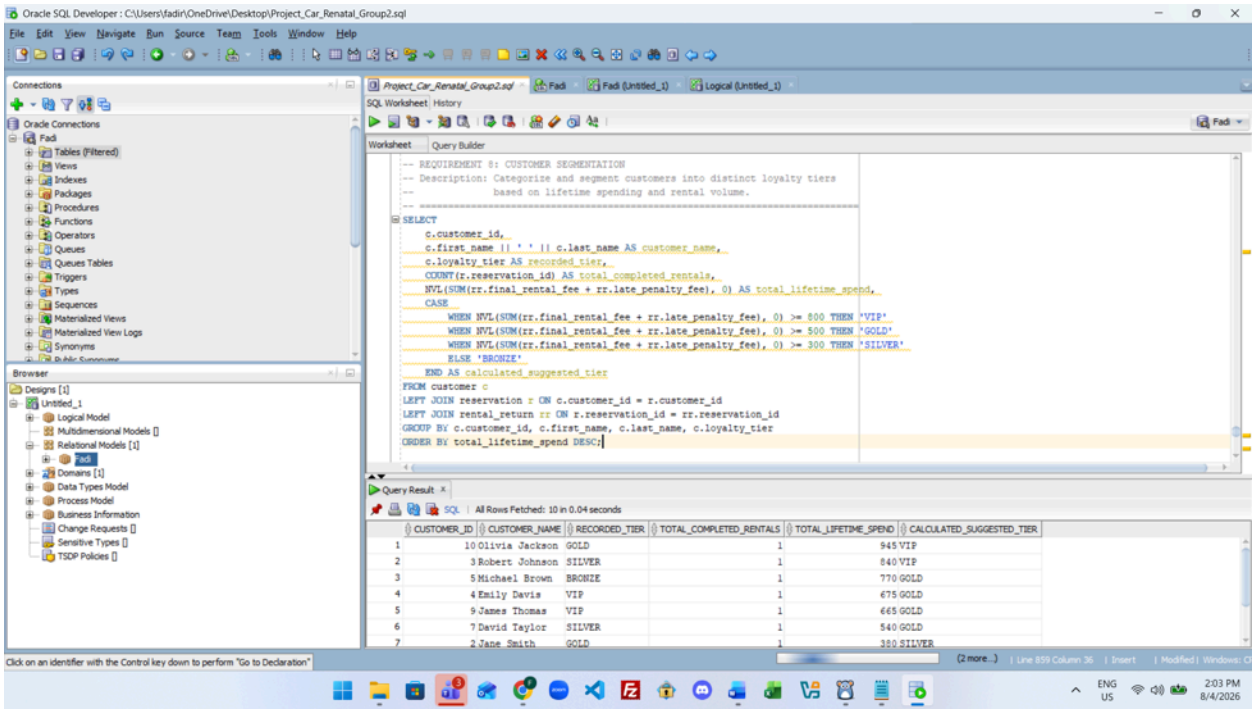
#7 INVENTORY ANALYTICS

Description: Identify the company's most popular or highly utilized vehicle model(s0 based on total rental transaction volumes.



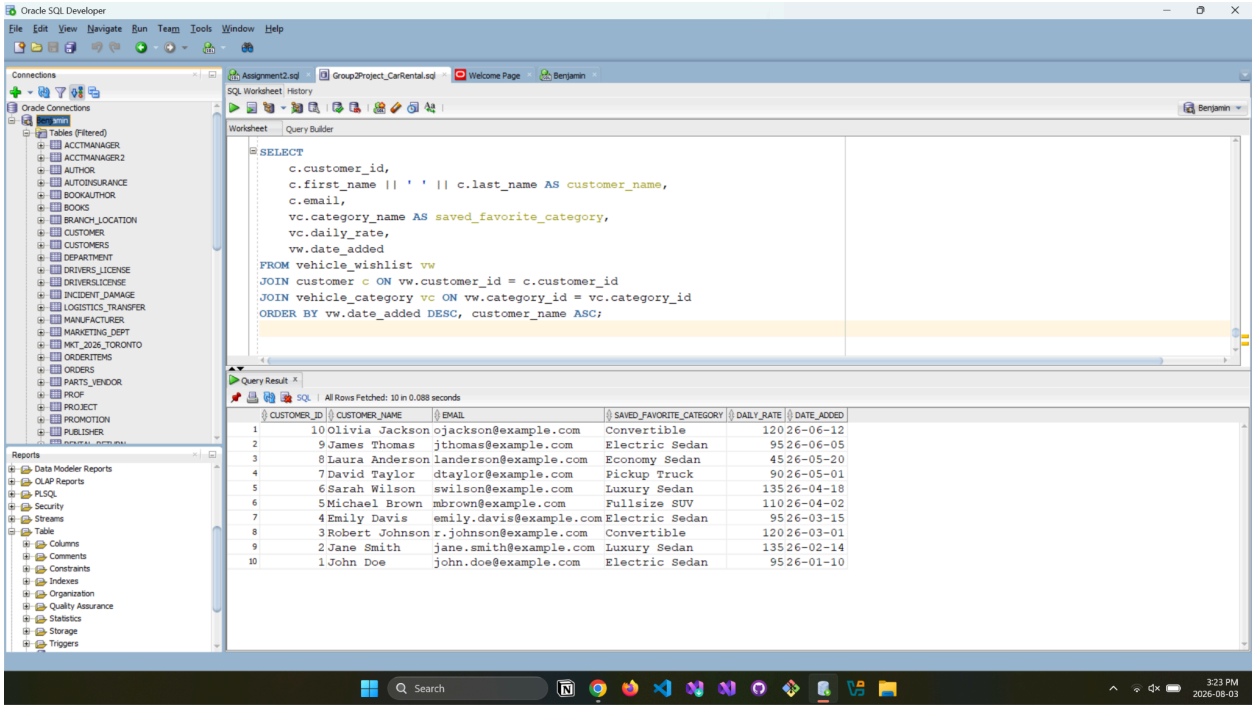
#8 CUSTOMER SEGMENTATION

Description: Categorize and segment customers into distinct loyalty tiers (e.g., Gold, VIP) based on their rental history and lifetime spending.



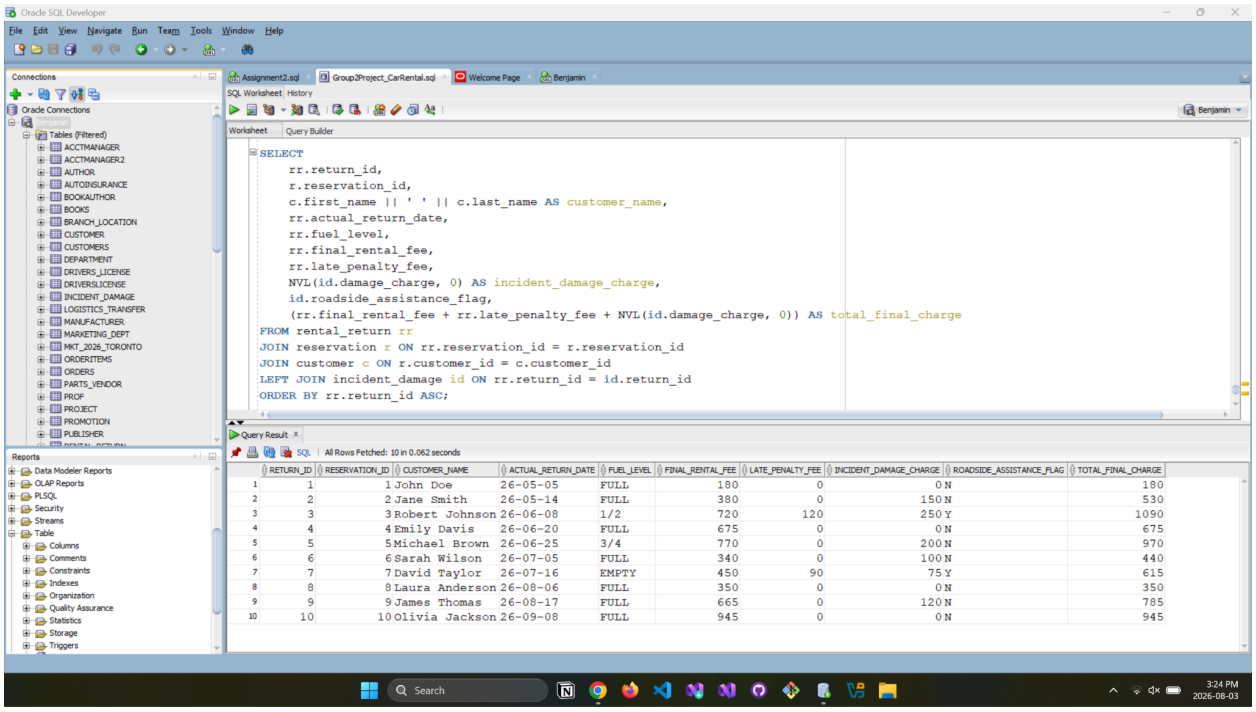
#9 WISH LIST ENGAGEMENT

Description: Extract and list saved or preferred vehicle classes that active users have added to their wishlist/favorites folder for future trips.



#10 CUSTOMER SERVICE PROCESSING

Description: Process transactional returns, handle roadside damage occurrences, and automate refund/incident fee calculations.



Project Conclusion: Velocity Car Rental Database Design

Project Overview: The Velocity Car Rental database project successfully architects and deploys an enterprise-grade relational database management system within an Oracle SQL environment, bridging theoretical design concepts with real-world operational demands.

Architecture & Security: By progressing through the database design lifecycle—from conceptual ER modeling to rigorous DDL/DML implementation—the system establishes a secure, scalable platform featuring strict referential integrity, automated surrogate keys, domain check validations, and role-based security.

Analytical Capability: The implementation of 10 complex analytical and operational business queries demonstrates the system's robust ability to handle advanced multi-table joins, aggregations, and executive reporting for modern car rental operations.

Normalization Status: The 12-table database schema achieves a 3rd Normal Form (3NF) baseline while fully satisfying Boyce-Codd Normal Form (BCNF) and 4th Normal Form (4NF).

BCNF Compliance: All functional dependencies across single candidate keys and multiple candidate keys—specifically within the CUSTOMER, VEHICLE, and RENTAL_RETURN entities—feature determinants that function as valid super keys.

4NF Evaluation: Multi-valued dependencies, including customer wish list items, driver licenses, and incident damages, are isolated into dedicated child relations to eliminate redundant multi-valued tuples.

Over-Normalization Rationale: Normalizing beyond 4NF into 5NF or DKNF was deliberately avoided to prevent excessive JOIN query latency, high CPU usage, and complex transactional overhead within this Online Transaction Processing (OLTP) environment.